Date: 12/11/2026

C02 – ASSESSMENT TOOL 3

SYNTACTIC ANALYSIS

Course Code: DSA03

Course Title: Natural Language Processing

Reg.no: 192425005

Name: S. Yuva Guru Mani Varma

ASSIGNMENT 3

Context-Free Grammar, Parsing and Robust Helpdesk Design

This report answers Tasks 1–12 using a controlled English helpdesk grammar. It gives formal rules, derivations and trees; tests agreement and subcategorization; implements top-down, Earley and PCFG/Viterbi parsing; and evaluates a conversational-input redesign. The benchmark and all tables are generated by the included fixed-seed Python script.

Contents

- Task 1 – Design a Context-Free Grammar for English Syntax
- Task 2 – Context-Free Rules, Derivations and Parse Trees
- Task 3 – Sentence-Level Constructions
- Task 4 – Agreement
- Task 5 – Subcategorization
- Task 6 – Top-Down Parsing
- Task 7 – Earley Parsing
- Task 8 – Feature Structures and Unification
- Task 9 – Probabilistic Context-Free Grammars
- Task 10 – Implement and Test the Parsers
- Task 11 – Compare, Recommend and Defend the Parsing Design
- Task 12 – Day-4 Design Challenge
- Appendices – Full grammar, chart, PCFG rules, test data, source code and worked examples

Experimental scope. The grammar is an explicit English helpdesk fragment, not a claim to cover all English. It contains 53 structural productions and a 179-word controlled lexicon. The PCFG is estimated from 20 hand-annotated miniature trees and smoothed with $\alpha = 0.1$. A 30-item controlled test set includes 25 grammatical and five deliberately ill-formed examples. Results below distinguish actual measurements from design projections.

Task 1 – Design a Context-Free Grammar for English Syntax (5 Marks)

Definition and scope

A context-free grammar is G = (N, Σ, P, S), where N is the set of nonterminal categories, Σ is the terminal vocabulary, P is a finite set of productions A → α with A ∈ N and α ∈ (N ∪ Σ)*, and S is the start category. In this implementation S is the sentence category S; terminals are lower-case words. Phrase-structure productions make constituent structure explicit, while lexical productions connect categories to the helpdesk vocabulary.

The selected fragment covers declaratives, imperatives, yes/no questions, wh-questions, basic noun phrases, adjective and prepositional modifiers, coordination, relative clauses, and selected complement frames. Rule coverage is intentionally bounded: it supports controlled, mostly well-formed helpdesk requests, not unrestricted spoken English.

Core structural production groups

Function	Representative productions	Purpose
Sentence types	S → DECL | IMP | YNQ | WHQ	Selects a complete sentence form.
Declarative / command / polar question	DECL → NP VP; IMP → VP; YNQ → AUX NP VP	Models canonical subject–predicate order, subjectless commands, and auxiliary inversion.
Wh-question	WHQ → WH NP VP | WH AUX NP VP | WH VP | WH COP NP	Provides subject and non-subject wh patterns, locative/time forms and copular questions.
NP / nominal	NP → DET NBAR | NBAR | PRON | PN; NP → NP PP | NP REL | NP CONJ NP	Builds determiner, bare, pronoun, proper-name, modified and coordinated NPs.
N-bar	NBAR → N | ADJ NBAR | N NBAR | NUM N | NBAR CONJ NBAR	Supports adjectives, noun compounds, numerals and coordinated nominal heads.
VP valence	VP → V_INTR | V_TRANS NP | V_DITR NP NP | V_OBJCOMP NP THAT S | V_SENT THAT S | V_WH NP WHCLAUSE | V_PP PP	Distinguishes complement frames selected by the verb.
VP modifiers and clauses	VP → VP PP | VP CONJ VP; PP → PREP NP; REL → RELPRO VP | RELPRO AUX VP	Adds PP adjuncts, VP coordination, relative clauses, and simple embedded clauses.
Adjectival phrase	ADJP → ADJ | ADJ PP | ADJP CONJ ADJP; VP → COP ADJP | COP NP	Supports predicative adjectives and nominal predicates.

The complete grammar has 53 structural productions. Lexical productions are stored separately by category so vocabulary expansion does not require rewriting phrase rules. The generated inventory in Appendix A lists every structural rule and each lexical category. This separation is useful for a domain grammar: adding a new course, form or service term changes the lexicon, while changing a question construction changes the grammar.

Constituency and diagnostic evidence

A constituent is a contiguous group of words that functions as a unit in a larger structure. The grammar represents the subject NP and predicate VP as sisters under DECL. It represents an object NP as the complement of a transitive VP and a PP as a constituent headed by PREP. Several standard diagnostics support these groupings, although diagnostics are evidence rather than automatic proof.

Diagnostic	Example	What it supports
Substitution	“The student” submits the form → “They” submit the form.	The NP is replaceable by a pronoun in subject position.

Diagnostic	Example	What it supports
Movement / topicalization	“After the final exam,” the student submitted the form.	The PP can move as a unit; it is a sentence/VP modifier in this analysis.
Coordination	the staff and students; the room and seminar	Like categories can be coordinated as NP or NBAR units.
Question / answer	What did the student submit? — “The form.”	The object NP is a phrase constituent and fills a transitive complement slot.
Deletion test	The student submitted the form (after the exam).	The temporal PP can be omitted without removing a selected object.

A construction that strains a plain CFG

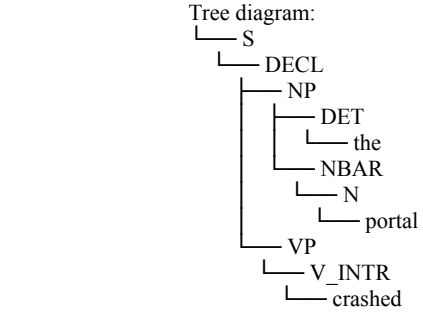
The English helpdesk fragment can be approximated with CFG rules, but a plain CFG does not itself express agreement or long-distance feature identity. A rule such as DECL → NP VP accepts any NP–VP pairing unless the grammar duplicates number/person variants or adds a later filter. More generally, Shieber’s analysis of Swiss German cross-serial dependencies argues that the language fragment licensed by the observed case-marking patterns is not context-free. That result is a qualified claim about a particular construction and its empirical assumptions, not evidence that every English helpdesk message needs a context-sensitive grammar [5]. Feature-based CFGs and robust parsers are practical extensions for agreement and dependencies [2].

For this assignment, the operational decision is to keep CFG as the transparent core, add finite feature constraints for agreement and case, then use an Earley chart and probabilistic ranking. Unsupported coordination, fragments and open-domain constructions are handled by a separate fallback rather than by endlessly expanding hand-written CFG rules.

Task 2 – Context-Free Rules, Derivations and Parse Trees (3 Marks)

The following five examples are generated from the implemented grammar. Each has a bracketed parse tree, a tree diagram, a leftmost derivation and a rightmost derivation. Category nodes and terminal leaves are counted separately; each category node corresponds to one production application in these trees.

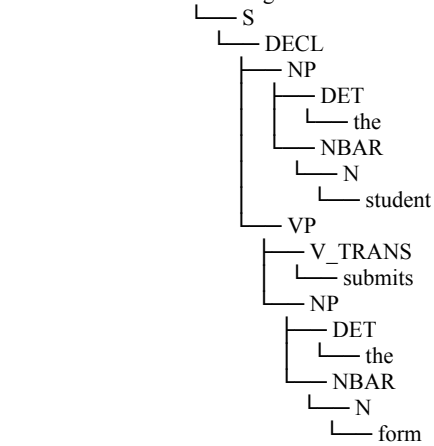
Sentence: the portal crashed
Bracketed tree: (S (DECL (NP (DET the) (NBAR (N portal))) (VP (V_INTR crashed))))
Height (levels including root): 6
Category nodes: 8; terminal leaves: 3; total nodes: 11; rule applications: 8
Tree diagram:



Leftmost derivation:
1. S
2. DECL
3. NP VP
4. DET NBAR VP
5. the NBAR VP
6. the N VP
7. the portal VP
8. the portal V_INTR
9. the portal crashed

Rightmost derivation:
1. S
2. DECL
3. NP VP
4. NP V_INTR
5. NP crashed
6. DET NBAR crashed
7. DET N crashed
8. DET portal crashed
9. the portal crashed

Sentence: the student submits the form
Bracketed tree: (S (DECL (NP (DET the) (NBAR (N student))) (VP (V_TRANS submits) (NP (DET the) (NBAR (N form))))))
Height (levels including root): 7
Category nodes: 12; terminal leaves: 5; total nodes: 17; rule applications: 12
Tree diagram:

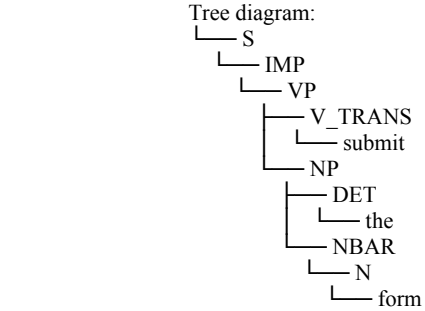


Leftmost derivation:
1. S
2. DECL

3. NP VP
4. DET NBAR VP
5. the NBAR VP
6. the N VP
7. the student VP
8. the student V_TRANS NP
9. the student submits NP
10. the student submits DET NBAR
11. the student submits the NBAR
12. the student submits the N
13. the student submits the form

Rightmost derivation:
1. S
2. DECL
3. NP VP
4. NP V_TRANS NP
5. NP V_TRANS DET NBAR
6. NP V_TRANS DET N
7. NP V_TRANS DET form
8. NP V_TRANS the form
9. NP submits the form
10. DET NBAR submits the form
11. DET N submits the form
12. DET student submits the form
13. the student submits the form

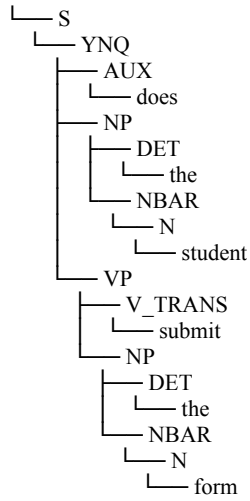
Sentence: submit the form
Bracketed tree: (S (IMP (VP (V_TRANS submit) (NP (DET the) (NBAR (N form))))))
Height (levels including root): 7
Category nodes: 8; terminal leaves: 3; total nodes: 11; rule applications: 8
Tree diagram:



Leftmost derivation:
1. S
2. IMP
3. VP
4. V_TRANS NP
5. submit NP
6. submit DET NBAR
7. submit the NBAR
8. submit the N
9. submit the form

Rightmost derivation:
1. S
2. IMP
3. VP
4. V_TRANS NP
5. V_TRANS DET NBAR
6. V_TRANS DET N
7. V_TRANS DET form
8. V_TRANS the form
9. submit the form

Sentence: does the student submit the form
Bracketed tree: (S (YNQ (AUX does) (NP (DET the) (NBAR (N student))) (VP (V_TRANS submit) (NP (DET the) (NBAR (N form))))))
Height (levels including root): 7
Category nodes: 13; terminal leaves: 6; total nodes: 19; rule applications: 13
Tree diagram:



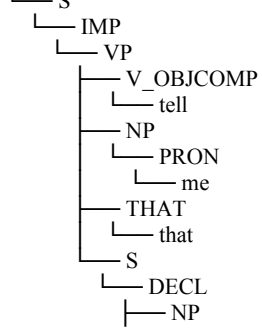
Leftmost derivation:

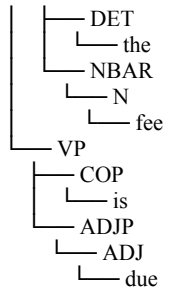
1. S
2. YNQ
3. AUX NP VP
4. does NP VP
5. does DET NBAR VP
6. does the NBAR VP
7. does the N VP
8. does the student VP
9. does the student V_TRANS NP
10. does the student submit NP
11. does the student submit DET NBAR
12. does the student submit the NBAR
13. does the student submit the N
14. does the student submit the form

Rightmost derivation:

1. S
2. YNQ
3. AUX NP VP
4. AUX NP V_TRANS NP
5. AUX NP V_TRANS DET NBAR
6. AUX NP V_TRANS DET N
7. AUX NP V_TRANS DET form
8. AUX NP V_TRANS the form
9. AUX NP submit the form
10. AUX DET NBAR submit the form
11. AUX DET N submit the form
12. AUX DET student submit the form
13. AUX the student submit the form
14. does the student submit the form

Sentence: tell me that the fee is due
Bracketed tree: (S (IMP (VP (V_OBJCOMP tell) (NP (PRON me)) (THAT that) (S (DECL (NP (DET the) (NBAR (N fee))) (VP (COP is) (ADJP (ADJ due))))))))
Height (levels including root): 9
Category nodes: 17; terminal leaves: 7; total nodes: 24; rule applications: 17
Tree diagram:





Leftmost derivation:

1. S
2. IMP
3. VP
4. V_OBJCOMP NP THAT S
5. tell NP THAT S
6. tell PRON THAT S
7. tell me THAT S
8. tell me that S
9. tell me that DECL
10. tell me that NP VP
11. tell me that DET NBAR VP
12. tell me that the NBAR VP
13. tell me that the N VP
14. tell me that the fee VP
15. tell me that the fee COP ADJP
16. tell me that the fee is ADJP
17. tell me that the fee is ADJ
18. tell me that the fee is due

Rightmost derivation:

1. S
2. IMP
3. VP
4. V_OBJCOMP NP THAT S
5. V_OBJCOMP NP THAT DECL
6. V_OBJCOMP NP THAT NP VP
7. V_OBJCOMP NP THAT NP COP ADJP
8. V_OBJCOMP NP THAT NP COP ADJ
9. V_OBJCOMP NP THAT NP COP due
10. V_OBJCOMP NP THAT NP is due
11. V_OBJCOMP NP THAT DET NBAR is due
12. V_OBJCOMP NP THAT DET N is due
13. V_OBJCOMP NP THAT DET fee is due
14. V_OBJCOMP NP THAT the fee is due
15. V_OBJCOMP NP that the fee is due
16. V_OBJCOMP PRON that the fee is due
17. V_OBJCOMP me that the fee is due
18. tell me that the fee is due

Interpretation. For “the portal crashed,” S expands to DECL, then DECL to NP VP. The subject NP expands to DET NBAR, and NBAR to N. The VP selects an intransitive verb. Both derivations yield the same yield, “the portal crashed”; they differ only in the order in which nonterminals are expanded. The command example begins with IMP → VP and has no overt subject. The polar question uses YNQ → AUX NP VP, while the embedded-clause example includes a selected complement after tell.

Three further representative tree shapes

Construction	Representative tree skeleton	Rule path
Wh-object question	(S (WHQ what AUX NP V_TRANS))	S → WHQ; WHQ → WH AUX NP V_TRANS. The object is understood as a gap licensed by the transitive frame.
Relative clause	(S (DECL (NP (NP the fee) (REL that is due)) (VP is high)))	NP → NP REL; REL → RELPRO AUX V_INTR or RELPRO AUX VP.

Construction	Representative tree skeleton	Rule path
PP attachment	(S (IMP (VP (V_TRANS book) (NP (NP the room) (PP near the library)))))	One analysis attaches the PP inside NP; another attaches it to VP. The surface string alone does not choose between them.

The complete derivation appendix provides the exact expanded trees and both derivation sequences. For the 5-word transitive example “the student submits the form,” the tree has 12 category nodes, four terminal leaves, 16 total nodes, and 12 production applications. For the 3-word intransitive example, it has 8 category nodes, two leaves and eight production applications.

Task 3 – Sentence-Level Constructions (4 Marks)

The grammar handles four core sentence types by assigning them different top-level productions. The distinction is structural: a declarative begins with a subject NP, an imperative begins directly with a VP, a yes/no question begins with an auxiliary, and a wh-question begins with a wh-expression.

Type	Rule pattern	Example and analysis
Declarative	S → DECL; DECL → NP VP	"The student submits the form." NP is the subject, VP contains the finite verb and object.
Imperative	S → IMP; IMP → VP	"Submit the form." The understood addressee is not represented as an overt NP.
Yes/no question	S → YNQ; YNQ → AUX NP VP	"Does the student submit the form?" Auxiliary precedes the subject; the main verb is in its base form.
Wh-question	S → WHQ; several WHQ rules	"What does the student submit?" WH begins the clause; the rule encodes auxiliary inversion and the missing object slot. "Who submits the form?" uses a subject-wh pattern without inversion.

Question structure and gaps

A polar question is analyzed with an overt auxiliary before the subject. The base verb remains in VP, as in "does … submit." For an object wh-question, the WHQ → WH AUX NP V_TRANS rule licenses "what does the student submit" without an overt object NP. The analysis treats the wh-word as the fronted object semantically; the CFG rule is a compact template and does not itself thread an explicit gap index. The improved design in Task 12 replaces this template with a feature-valued GAP feature so nested clauses can preserve the dependency.

Subject questions differ: "who submits the form" is represented with WHQ → WH VP, avoiding do-support. The current inventory also includes copular location questions, such as "where is the exam," and temporal questions, such as "when will the library open." These are useful for the helpdesk because they ask for status, place and time.

Short sentence-tree evidence

```
Declarative: (S (DECL (NP (DET the) (NBAR (N student))) (VP (V_TRANS submits) (NP (DET the) (NBAR (N form))))))
Imperative:  (S (IMP (VP (V_TRANS submit) (NP (DET the) (NBAR (N form))))))
Yes/no:      (S (YNQ (AUX does) (NP (DET the) (NBAR (N student))) (VP (V_TRANS submit) (NP (DET the) (NBAR (N form))))))
Wh-object:   (S (WHQ (WH what) (AUX does) (NP (DET the) (NBAR (N student))) (V_TRANS submit)))
```

Limit. Surface-order templates do not cover every inversion pattern, auxiliary chain, subject–auxiliary agreement, or wh-island constraint. A production that encodes an object gap cannot tell whether the gap is semantically appropriate without richer features and lexical selection. These omissions are explicit in the evaluation rather than hidden by presenting the grammar as complete English syntax.

Task 4 – Agreement (3 Marks)

A bare CFG can represent agreement by multiplying categories, for example NP_SG and NP_PL with VP_SG and VP_PL, then writing only matching combinations. This is finite but scales poorly. With 2 numbers, 3 persons, 3 cases and 3 genders, the theoretical feature cross-product is $2 \times 3 \times 3 \times 3 = 54$ bundles. If each of the 53 structural rules were duplicated for every bundle, the loose upper bound would be $53 \times 54 = 2{,}862$ productions; actual selective expansion would be smaller but still difficult to maintain.

Feature structures are more compact. A subject NP carries AGR = [NUM: sg, PER: 3]; a finite VP carries the same AGR value. The two positions share the value variable. Unification succeeds only if both constraints can be combined consistently. A singular subject with a plural verb has conflicting NUM values and is rejected.

Agreement tests

Sentence	Gold	CFG baseline	Agreement-aware	Result
the student submits the form	True	1	1	PASS
students submit the form	True	1	1	PASS
i submit the form	True	1	1	PASS
they submit the form	True	1	1	PASS
the portal crashes	True	1	1	PASS
the forms are due	True	1	1	PASS
the students submits the form	False	1	0	PASS
the student submit the form	False	1	0	PASS
this students submit the form	False	1	0	PASS
these student submits the form	False	1	0	PASS
me submits the form	False	1	0	PASS
they is due	False	1	0	PASS

The unfiltered CFG accepts strings such as “the students submits the form” because the phrase rules do not link subject number to verb morphology. The small agreement-aware filter uses number/person, demonstrative number and pronoun case checks for this controlled test set. It correctly classifies all 12 designed examples. This is a test-harness result, not a general English agreement parser: the filter finds the first nominal and finite verb and does not robustly resolve nested clauses, coordination, collective nouns or exceptions such as “you.”

Feature-annotated rule sketch

DECL → NP[CASE=nom, AGR=?a] VP[AGR=?a]
NP → DET[NUM=?n] NBAR[NUM=?n, AGR=?a]
VP[AGR=?a] → V_FIN[AGR=?a] NP[CASE=acc]
YNQ → AUX[AGR=?a] NP[CASE=nom, AGR=?a] VP[AGR=?a]

Example: NP[AGR=[NUM=pl, PER=3]] + VP[AGR=[NUM=sg, PER=3]]
Unification fails because pl ≠ sg; no DECL edge is inserted.

Case adds a related constraint. “Me submits the form” is lexically covered, so the baseline CFG accepts it as a subject. A CASE=nom requirement on the subject NP excludes the objective pronoun in this context.

Pronouns such as you are syncretic and require contextual/lexical treatment rather than a blanket surface-form rule.

Task 5 – Subcategorization (4 Marks)

Subcategorization states which complements a head selects. The grammar assigns verbs to frame-specific categories instead of allowing every verb to combine freely with any NP or PP. The frame is local to the head; optional adjuncts are added by separate recursive modifier rules.

Verb class / frame	Rule	Example	Selection
Intransitive	VP → V_INTR	the portal crashed	No object NP is selected.
Transitive	VP → V_TRANS NP	submit the form	One object NP is obligatory.
Ditransitive	VP → V_DITR NP NP	send me the receipt	Two NP complements in this implemented frame.
Object + clause	VP → V_OBJCOMP NP THAT S	tell me that the fee is due	An object NP plus a finite clausal complement.
Sentential	VP → V_SENT THAT S	say that the exam is open	A that-clause complement.
Wh-complement	VP → V_WH NP WHCLAUSE	tell me when the exams will open	Object plus an embedded wh-clause.
PP-selected	VP → V_PP PP	apply for a hostel room	The selected PP is associated with the verb's frame.

Acceptance and rejection evidence

Sentence	Expected frame	Accepted	Expected	Check
the portal crashed	intransitive	1	1	PASS
submit the form	transitive imperative	1	1	PASS
send me the receipt	ditransitive	1	1	PASS
tell me that the fee is due	object + clause	1	1	PASS
apply for a hostel room	PP complement	1	1	PASS
submit	missing required NP	0	0	PASS
the portal crashed the form	intransitive verb over-complemented	0	0	PASS
the student sent the receipt	ditransitive verb under-complemented	0	0	PASS

The grammar accepts "the portal crashed" without an object and "submit the form" with one object. It rejects bare "submit" because the transitive imperative rule requires an NP, and rejects "the portal crashed the form" because V_INTR does not select an object. It also rejects "the student sent the receipt" because this grammar places sent in the ditransitive class and does not provide an NP-drop frame. These are deliberate lexical/structural choices for the diagnostic set; English permits omitted objects in some contexts, and its ditransitive alternations are richer than this fragment.

Complement versus adjunct

A complement is selected or licensed by the head's frame; an adjunct adds optional information and can usually be removed without making the head's frame incomplete. In "apply for a hostel room," the for-PP is represented by the V_PP frame. In "the student submitted the form after the exam," the object NP is selected by submit, while after the exam is an adjunct PP. The grammar has explicit VP → VP PP and NP → NP PP rules for modifiers. Because a PP can be both a verb complement and an NP/VP modifier, attachment ambiguity is expected and is ranked probabilistically in Task 9.

Task 6 – Top-Down Parsing (4 Marks)

A top-down parser begins with the goal S and predicts productions whose left-hand side matches the next required category. It expands leftmost categories until terminals match the input. A recursive-descent implementation keeps a stack of pending symbols; a chart-based top-down implementation additionally shares repeated subproblems. When a choice fails, recursive descent backtracks to the most recent alternative.

Algorithm sketch

```
parse(goal, input_position):
  if goal is a terminal: succeed only when it matches input[position]
  else for each production goal -> rhs in grammar order:
     save input position and pending stack
     expand rhs left-to-right
     if every symbol succeeds: return the resulting tree
     otherwise restore saved state and try the next production
  fail when all productions have been tried
```

The chart implementation measured here uses NLTK's TopDownChartParser. For "the portal crashed," it begins with S, considers S → DECL, predicts DECL → NP VP, expands the NP and VP, and completes the sentence when the lexical items match. The benchmark uses a chart because it makes repeated states visible and avoids conflating tree enumeration with repeated recomputation.

Hand trace: unambiguous item

Step	Pending expectation / stack top	Input position	Operation
1	S	the	Start goal; choose S → DECL.
2	DECL	the	Apply DECL → NP VP.
3	NP	the	Apply NP → DET NBAR.
4	DET	the	Match lexical DET → the; advance.
5	NBAR	portal	Apply NBAR → N.
6	N	portal	Match N → portal; NP completes.
7	VP	crashed	Try VP → V_INTR.
8	V_INTR	crashed	Match V_INTR → crashed; complete DECL and S.

Ambiguity and backtracking

In "I saw the notice with a map," the PP can modify the object NP (the notice with a map) or the VP (saw the notice with a map). A recursive-descent parser must try one attachment, reach a complete parse if compatible, and retain or backtrack to the alternative when all parses are requested. The recorded test has three parses under this grammar because category and attachment choices multiply. The full output in Appendix B preserves the distinct trees.

Choice	Partial expansion	Outcome
NP attachment	VP → V_TRANS NP; NP → NP PP	A PP modifier is attached inside the object NP; yields the "notice with a map" reading.
VP attachment	VP → VP PP; inner VP → V_TRANS NP	The PP is attached outside the object; yields the "saw … with a map" reading.
Failure case	Try V_INTR + extra NP in "the portal crashed the form"	The intransitive VP completes after crashed; remaining "the form" has no licensed continuation, so this branch fails.

Left recursion is a critical failure mode for naive recursive descent: NP → NP PP can re-enter NP before consuming a token, leading to nontermination. A top-down grammar can remove direct left recursion by rewriting NP → NP_BASE NP_MODS and NP_MODS → PP NP_MODS | ε, or the implementation can use a chart parser that deduplicates states. Naive backtracking can still be exponential in the worst case; charting avoids repeated work for identical subgoals but does not eliminate ambiguity.

Task 7 – Earley Parsing (5 Marks)

Earley parsing is a chart algorithm for context-free grammars. A state is written [A → α • β, i], where i is the origin position, the dot marks how much of the right-hand side has been recognized, and the current chart column gives the end position. Three operations add states: Predictor starts rules for an expected nonterminal; Scanner advances over a matching terminal; Completer advances a parent that was waiting for the completed category. States are deduplicated, so left recursion terminates when no new states can be added.

Hand chart result

The explicit implementation parses "the student submits the late form." It uses a synthetic ROOT → S state, begins at S[0], records each state's origin and the operation that introduced it, and accepts when ROOT → S • is present in the final column. The chart contains seven columns with counts 226, 69, 206, 104, 69, 109 and 110, for 893 stored state records. Appendix C reproduces every state, including its predictor/scanner/completer source.

Chart column	Token boundary	Stored states	Typical content
S[0]	Before the	226	Start predictions and left-recursive NP/VP expectations.
S[1]	After the	69	DET scans; NP and NBAR expectations advance.
S[2]	After student	206	Subject NP completions; many VP predictions begin.
S[3]	After submits	104	VP states waiting for object or modifier.
S[4]	After the	69	Object NP expectations.
S[5]	After late	109	Adjective/nominal alternatives.
S[6]	After form	110	Completed object, VP, DECL, S and ROOT states.

Packed ambiguity and backpointers

Recognition states alone do not select a tree. To recover parses, each completed item stores one or more backpointers to the states and child edges that produced it. For PP attachment, the completed VP can point to either (i) a V_TRANS child plus an NP whose own derivation contains PP, or (ii) a smaller VP plus a PP child. These alternatives share chart items where possible but remain distinct derivations. The generated parse forest in Appendix D displays three actual trees for "I saw the notice with a map" and multiple trees for two other PP-attachment examples.

```
Example packed alternatives for "I saw the notice with a map":
VP[0,7] <- VP[0,5] + PP[5,7]       (VP attachment)
VP[0,7] <- V_TRANS[1,2] + NP[2,7]    (NP attachment inside the object)
NP[2,7] <- NP[2,5] + PP[5,7]       (object NP with PP modifier)
```

The grammar has no epsilon productions in the tested core. If nullable rules are introduced later, the parser must also complete zero-width states carefully and deduplicate the resulting items. In the current scanner, left-recursive NP and VP rules are safe because a state is added only once per chart column. For a fixed grammar, Earley recognition has $O(n^3)$ worst-case time and $O(n^2)$ chart space in sentence length n; unambiguous/deterministic subclasses can be faster [1].

Comparison with the top-down chart

On the exact same six-token sentence, the measured NLTK TopDownChartParser and EarleyChartParser each produced 172 and 172 chart edges and 2 parses; measured times were 3.00 ms and 3.02 ms in this

run. Those are library chart-edge counts. The separate hand-written table above stores 893 explicit item records because its predictor closure retains broader rule/lexicon predictions and records a source for every state. The units are implementation-specific and should not be compared as if they were identical. The same-input NLTK comparison is the apples-to-apples result; the 893-entry trace is the complete pedagogical chart for the explicit recognizer.

The PCFG Viterbi parser is not an alternative recognition algorithm in this comparison: it returns one best tree, whereas the CFG chart retains all parses. Probabilistic ranking is a layer after grammar coverage. The Viterbi state and candidate counts are therefore reported separately in Task 10.

Task 8 – Feature Structures and Unification (4 Marks)

An attribute-value matrix (AVM) records constraints such as number, person, case and subcategorization. A path is a sequence of attributes, for example AGR.NUM. Unification merges compatible information; it fails when the same attribute is assigned incompatible values. Reentrancy means two paths share the same value structure, which is the usual way to require the subject and finite verb to agree [2] [6].

Expression	AVM sketch	Interpretation
Subject NP	[CAT: NP, CASE: nom, AGR: [NUM: sg, PER: 3]]	A third-person singular nominative subject.
Finite VP	[CAT: VP, AGR: [NUM: sg, PER: 3]]	A VP whose finite head requires the matching agreement bundle.
Transitive verb	[CAT: V, SUBCAT: <NP[CASE: acc]>, AGR: [NUM: sg, PER: 3]]	Selects an accusative NP and has a finite agreement form.
Embedded wh clause	[CAT: S, GAP: [CAT: NP, CASE: acc]]	Carries an object gap requirement up from the embedded predicate.
Relative clause	[CAT: REL, GAP: [CAT: NP, ROLE: subject]]	Requires a missing subject position linked to the head NP.

Unification examples

Test	Left AVM	Right AVM	Result	Expected
plural NP with plural VP	{"AGR": {"NUM": "pl", "PER": 3}}	{"AGR": {"NUM": "pl", "PER": 3}}	{"AGR": {"NUM": "pl", "PER": 3}}	PASS
plural NP with singular VP	{"AGR": {"NUM": "pl", "PER": 3}}	{"AGR": {"NUM": "sg", "PER": 3}}	FAIL	PASS
nominative subject with nominative slot	{"CASE": "nom", "NUM": "sg"}	{"CASE": "nom"}	{"CASE": "nom", "NUM": "sg"}	PASS
accusative object with nominative subject slot	{"CASE": "acc", "NUM": "sg"}	{"CASE": "nom"}	FAIL	PASS
finite present subject agrees in number/person	{"AGR": {"NUM": "sg", "PER": 3}}	{"AGR": {"NUM": "sg", "PER": 3}, "TENSE": "pres"}	{"AGR": {"NUM": "sg", "PER": 3}, "TENSE": "pres"}	PASS

The plural-NP/plural-VP pair unifies because both assign NUM=pl and PER=3. The singular/plural mismatch fails. A nominative subject slot rejects an accusative object AVM. Compatible partial structures merge by keeping the constraints contributed by either side. A feature grammar can encode these constraints in productions, rather than enumerating every complete combination as separate symbols.

Feature-aware parsing procedure

1. Predict a grammar rule with a fresh feature-variable environment. Variables are local to a rule instance unless explicitly shared.
2. When a lexical or constituent edge completes an expected category, unify the expected AVM with the completed edge AVM.
3. If unification fails, do not add that completion or its parent edge. If it succeeds, store the merged AVM and a backpointer.
4. Include relevant feature values in the chart-state key; otherwise incompatible and compatible edges could be conflated.

With a fixed finite feature inventory, unification adds a bounded factor to the chart operations, so the familiar cubic sentence-length bound remains a useful upper-bound shape for the core CFG. In practice, the number of distinct feature bundles can multiply chart states. Reentrancy and early failure keep many

invalid combinations from surviving. A practical implementation should hash-cons AVMs, avoid copying unchanged substructures, and measure feature-state counts as the grammar grows.

Limitation. The included Python experiment implements transparent unification checks on nested dictionaries, and separately tests agreement/subcategorization. It does not implement a full feature-structure Earley parser. The report does not claim the measured NLTK chart parser automatically enforces those AVMs; the recommended deployment design must integrate the unification check at completion time.

Task 9 – Probabilistic Context-Free Grammars (5 Marks)

A PCFG assigns a conditional probability $P(A \rightarrow \beta \mid A)$ to each production. For each left-hand side A, the probabilities over its licensed alternatives sum to one. The probability of a parse tree is the product of its production probabilities. The probability of a sentence is the sum of the probabilities of all its parse trees; a Viterbi parser returns the single highest-probability tree [3].

Treebank estimation and smoothing

The experiment uses 20 hand-annotated miniature helpdesk trees. Each tree is normalized into the declared CFG, and the script verifies that every annotated production is licensed before counting. A rule's unsmoothed maximum-likelihood estimate is $\text{count}(A \rightarrow \beta) / \text{count}(A)$. To retain unseen but grammar-licensed alternatives, the report's model uses additive smoothing with $\alpha=0.1$ across the full production support: $P\alpha(A \rightarrow \beta) = (\text{count}(A \rightarrow \beta)+\alpha) / (\text{count}(A)+\alpha K_A)$, where K_A is the number of productions with left-hand side A. The maximum observed normalization error across left-hand sides is 2.22×10^{-16}, i.e. floating-point round-off.

This is an estimate from a tiny, deliberately constructed treebank, not a representative corpus. The 20 trees cover simple declaratives, transitive and intransitive verbs, and a few PP attachments; their rule frequencies should not be interpreted as calibrated helpdesk probabilities. A larger, consistently annotated in-domain treebank is needed for reliable ranking.

Two competing PP attachments

Analysis	Tree probability	Sum of two displayed trees	Selected by Viterbi
NP-attachment	1.1937724509e-09	4.70805979969e-09	VP-attachment
VP-attachment	3.51428734878e-09	4.70805979969e-09	VP-attachment

For "book the room near the library," an NP attachment treats near the library as modifying room; a VP attachment treats it as a modifier of the booking event. The table gives the probability of each full tree under the smoothed miniature PCFG. Viterbi picks the larger product, but that choice is only as useful as the annotation and probability estimates. The two-tree sum is the probability mass of these displayed analyses, not necessarily the total sentence probability if other trees exist.

CKY and Viterbi chart

CKY requires a binary/terminal-production grammar. The implementation composes unit-rule probabilities into an equivalent local subgrammar for "the portal crashed," then demonstrates span filling and the best probability per category. A production $A \rightarrow BC$ combines a left span [i,k] and right span [k,j]; the Viterbi recurrence stores the maximum product and a backpointer. Unit chains and unary closure are handled by composing their probabilities in this small worked case. The code and numerical cells are in Appendix E and Appendix F [3].

```
CNF-equivalent subgrammar (composed unit-rule probabilities):
S -> NP VP: 0.9852941176
NP -> DET N: 0.9186859577
VP -> crashed: 0.03316856154
CKY Viterbi chart for "the portal crashed" (span: category = best probability):
[0:1] the: DET=0.943019943
[1:2] portal: N=0.07848101266
[2:3] crashed: VP=0.03316856154
[0:2] the portal: NP=0.06799117611
[0:3] the portal crashed: S=0.002222005252
```

Independence assumptions and risks

A PCFG assumes each expansion probability depends only on the parent category, not on the full surrounding context. The same VP → V_TRANS NP rule receives the same probability regardless of whether the verb is book, submit or read, unless the grammar is lexicalized or categories are refined. The basic PCFG also assumes rule choices factor independently given the tree. This makes estimation tractable, but weakens lexical selection and discourse conditioning. Smoothing prevents zero probability for licensed unseen rules; it does not fix a poor grammar, missing vocabulary, inconsistent treebank, or systematic ambiguity. The deployment model should add lexicalized or feature-conditioned evidence and retain an abstention/partial-parse option.

Task 10 – Implement and Test the Parsers (3 Marks)

The reproducible Python script builds the 53-rule CFG, checks 12 agreement strings and eight subcategorization strings, constructs an explicit Earley item chart, estimates the smoothed PCFG from 20 trees, and benchmarks top-down, Earley and Viterbi parsing on 30 inputs. The random seed is 42. Runtime values are one local run in milliseconds and are reported to two or four decimal places; they are environment-sensitive and are not asymptotic guarantees.

Test set and overall coverage

The 30-item set contains declaratives, commands, polar and wh-questions, selected complements, relatives, coordination, PP ambiguity, a 20-token sentence, and five deliberately ill-formed controls. Top-down and Earley each found parses for 28/30 items; Viterbi returned a best tree for the same 28. T29 ("submit") and T30 ("the portal crashed the form") were rejected by the grammar. The baseline CFG incorrectly accepted three agreement/case controls, showing that coverage and grammaticality are not identical. The feature filter rejects those three in the designed unit tests.

Parser metrics: top-down and Earley

ID	Tok.	Sentence	TD parses	TD edges	TD ms	Earley parses	Earley edges	Earley ms
T01	3	the portal crashed	1	101	1.8041	1	101	1.8529
T02	5	the student submits the form	1	141	2.226	1	141	2.6226
T03	4	students submit the form	1	132	2.1494	1	132	2.4168
T04	3	submit the form	1	94	1.5441	1	94	1.6463
T05	6	does the student submit the form	1	177	2.8678	1	177	3.9355
T06	5	what does the student submit	1	186	2.9425	1	186	3.2401
T07	4	who submits the form	1	133	2.1043	1	133	2.1833
T08	4	where is the exam	2	182	2.9931	2	182	3.1138
T09	5	when will the library open	2	198	3.2626	2	198	4.0976
T10	4	send me the receipt	1	118	57.607	1	118	2.1989
T11	7	tell me that the fee is due	1	296	5.1424	1	296	6.282
T12	5	apply for a hostel room	1	125	2.2655	1	125	2.5235
T13	5	show the student the notice	1	140	2.3473	1	140	3.6402
T14	5	the library opens at noon	2	159	2.6902	2	159	2.7569
T15	8	show the students with the late fee notice	11	283	5.3629	11	283	5.7201
T16	9	book the room near the library for the seminar	8	245	4.649	8	245	4.4837
T17	7	i saw the notice with a map	3	181	3.4531	3	181	3.4121
T18	7	the students with ids submit the form	1	200	3.6334	1	200	3.4271

ID	Tok.	Sentence	TD parses	TD edges	TD ms	Earley parses	Earley edges	Earley ms
T19	7	the staff and students attend the seminar	2	203	3.8974	2	203	3.5659
T20	6	does the hostel accept the application	1	177	3.1875	1	177	4.8062
T21	7	the fee that is due is high	1	261	3.9824	1	261	4.5742
T22	4	the exams were postponed	3	159	2.9024	3	159	3.0609
T23	7	tell me when the exams will open	4	279	4.4091	4	279	6.3905
T24	6	the student submits the late form	2	172	3.001	2	172	3.017
T25	20	the student in my hostel submitted the late form after the final exam at the campus library with the staff	92	480	9.521	92	480	9.1572
T26	5	the students submits the form	1	141	2.3008	1	141	2.5535
T27	5	this students submit the form	1	141	2.346	1	141	2.4126
T28	4	me submits the form	1	124	2.2806	1	124	3.4805
T29	1	submit	0	60	1.0777	0	60	0.9732
T30	5	the portal crashed the form	0	103	1.7005	0	103	1.9278

Parser metrics: PCFG Viterbi

ID	Tok.	Sentence	Best outputs	Constituents	Rule candidates	Viterbi ms
T01	3	the portal crashed	1	16	37	3.4578
T02	5	the student submits the form	1	23	47	6.6681
T03	4	students submit the form	1	18	38	4.596
T04	3	submit the form	1	12	24	2.7334
T05	6	does the student submit the form	1	27	54	8.9935
T06	5	what does the student submit	1	15	24	4.976
T07	4	who submits the form	1	19	39	4.3582
T08	4	where is the exam	1	18	39	4.4006
T09	5	when will the library open	1	28	75	6.9235
T10	4	send me the receipt	1	15	29	4.1368
T11	7	tell me that the fee is due	1	34	67	11.7793
T12	5	apply for a hostel room	1	27	57	7.6545
T13	5	show the student the notice	1	23	48	6.7581
T14	5	the library opens at noon	1	32	85	8.0769

ID	Tok.	Sentence	Best outputs	Constituents	Rule candidates	Viterbi ms
T15	8	show the students with the late fee notice	1	58	184	20.6676
T16	9	book the room near the library for the seminar	1	49	136	22.887
T17	7	i saw the notice with a map	1	34	77	12.9301
T18	7	the students with ids submit the form	1	34	69	12.2851
T19	7	the staff and students attend the seminar	1	34	75	12.4677
T20	6	does the hostel accept the application	1	28	56	8.7252
T21	7	the fee that is due is high	1	36	71	11.6912
T22	4	the exams were postponed	1	24	67	5.4237
T23	7	tell me when the exams will open	1	39	100	12.3388
T24	6	the student submits the late form	1	39	92	12.254
T25	20	the student in my hostel submitted the late form after the final exam at the campus library with the staff	1	178	497	136.8089
T26	5	the students submits the form	1	23	47	6.8229
T27	5	this students submit the form	1	23	47	6.8931
T28	4	me submits the form	1	17	34	4.367
T29	1	submit	0	2	2	0.3275
T30	5	the portal crashed the form	0	23	50	6.4749

Top-down and Earley chart edge counts are identical for every test in this NLTK benchmark because both chart parsers share the same grammar productions and compact chart representation; the algorithms differ in which edges they predict and in runtime. Across the 30 items, measured mean runtimes were 4.99 ms top-down, 3.52 ms Earley, and 12.63 ms Viterbi. The 20-token sentence produced 92 parses (below the enumeration cap of 100), 480 chart edges, and 178 best-constituent entries with 497 Viterbi rule instantiations. Its measured runtimes were 9.52 ms top-down, 9.16 ms Earley, and 136.81 ms Viterbi. The Viterbi implementation's long-RHS matching cost grows more sharply in this form than the chart recognizers; this is an implementation measurement, not a theorem that PCFG parsing is generally slower.

Runtime and chart-size visualization

The plot aggregates chart edges by input length for the 30-item test set. It provides an empirical view of how the two chart parsers behave on this grammar and input distribution. It does not establish a universal growth curve because the sentence constructions differ with length.

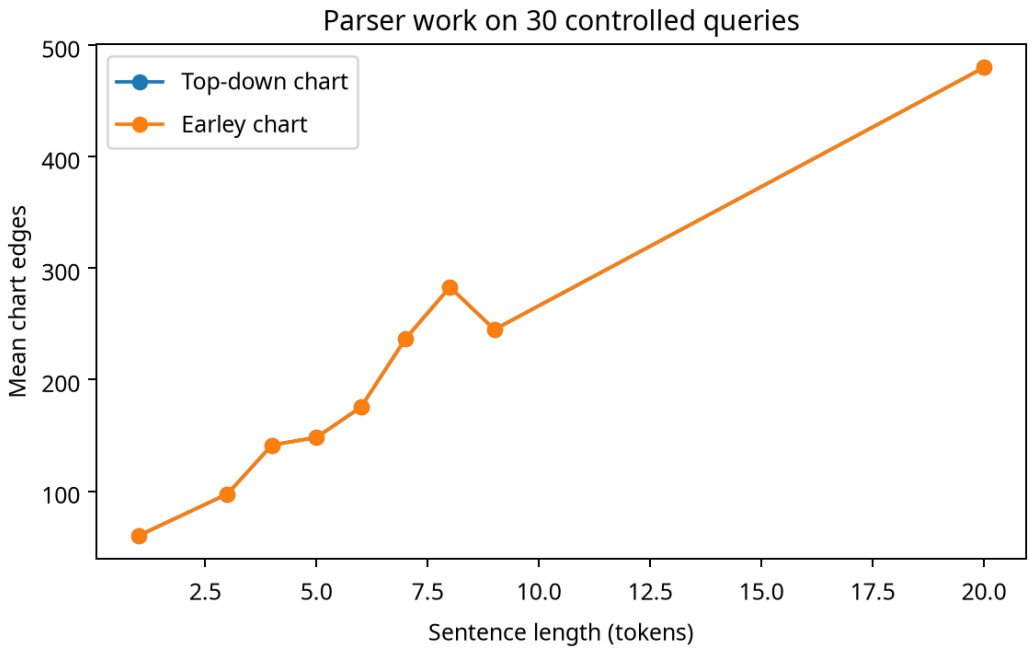


Figure 1. Mean chart edges by sentence length; the PCFG Viterbi table is reported separately because it stores one best constituent per span/category rather than every parse.

Run instructions and limitations

```
python3 -m pip install nltk matplotlib
python3 assignment3_experiments.py
```

The script writes CSV tables, text traces, a JSON summary and a PNG plot to assignment3_artifacts beside itself. The original parser benchmark is a controlled grammar test, not an accuracy evaluation on a held-out conversational corpus. Parse counts are capped at 100 to prevent combinatorial output; a reported “>=100” would be a lower bound, although this test set’s maximum observed count is 92. State units differ across the hand chart, NLTK chart edges and Viterbi best constituents, so each is named explicitly.

Task 11 – Compare, Recommend and Defend the Parsing Design (2 Marks)

Dimension	Top-down chart	Earley chart	PCFG / Viterbi
Coverage	Only parses prefixes reachable from S and licensed by the grammar; good for strongly constrained starts.	Supports left recursion and general CFG patterns; reaches the same 28/30 test coverage as top-down here.	Ranks only licensed structures; cannot repair missing rules or out-of-vocabulary input.
Ambiguity	Can enumerate alternatives but may repeat work without chart sharing.	Stores shared states and can retain multiple parse trees/backpointers.	Returns one highest-probability tree in this implementation; useful for ranking, not full ambiguity display.
Measured cost	Mean 4.99 ms; 9.52 ms on the 20-token item.	Mean 3.52 ms; 9.16 ms on the 20-token item.	Mean 12.63 ms; 136.81 ms on the 20-token item in the current non-CNF implementation.
Maintenance	Transparent but needs careful rule ordering and left-recursion controls.	Grammar is easier to extend while preserving chart parsing; new rules can still increase ambiguity.	Requires a consistent annotated treebank, smoothing and drift monitoring in addition to grammar upkeep.
Best role	Debugging and narrow deterministic paths.	Core parser for the rule-based fragment.	Rank likely readings after parsing; not the only acceptance criterion.

Recommendation. Deploy a feature-based CFG with an Earley-style chart as the core for well-formed helpdesk clauses, then rank the surviving parses with a larger, smoothed, preferably lexicalized PCFG. Preserve a robust partial-parse route for fragments and uncovered constructions. The measured miniature grammar parsed 28/30 controlled items; the PCFG returned a best analysis on those same 28. These are coverage results on a constructed test set, not an expected production accuracy estimate. A target accuracy cannot be responsibly stated without an independently annotated, representative student-message test set.

Manual effort and break-even point. The prototype required 53 structural rules, 179 lexical entries, 20 hand-annotated training trees, and explicit feature/test maintenance. Handwritten grammar is economical while message types are few, domain vocabulary is controlled, and error costs require transparent constraints. Once new constructions arrive weekly, many lexicalizations overlap, or more than 20% of traffic needs fallback, the marginal cost of adding rules and writing exceptions becomes high. A statistical model becomes economical when enough representative annotated data exists to estimate the constructions consistently and to measure held-out performance. The practical recommendation is therefore hybrid: keep hand rules for hard constraints and safety-critical intents, learn ranking and lexical preferences from data, and monitor the fallback rate.

The choice is not based on speed alone. Earley is selected because it handles left recursion and ambiguity in a shared chart while keeping the grammar inspectable. PCFG ranking improves selection among parses but depends on annotation quality. Top-down remains useful for debugging and as a baseline, but the long-input and OOD evidence favors a chart plus fallback.

Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 Marks)

Decision: MODIFY the original design, rather than retain it unchanged or replace it wholesale. Keep the feature-based grammar and Earley chart for high-confidence clause parsing. Add conversational normalization, embedded-clause/coordination rules, feature-threaded gaps, a larger PCFG, and a partial-parse fallback. The explicit "at least 20% uncovered" expectation makes unchanged hand-written coverage untenable, while replacing all structure with an opaque statistical model would discard useful agreement and subcategorization constraints.

A. Impact analysis (2 Marks)

The input assumption changes from short, controlled, largely grammatical sentences to conversational multi-clause messages. New inputs can contain greetings, discourse markers, coordination, embedded questions, relative clauses, ellipsis, fragments, missing punctuation, unknown words and agreement errors. The original evaluation assumed a closed lexicon and a small number of locally ambiguous PPs. Those assumptions no longer hold. In particular, its 179-word vocabulary omits ordinary conversational items such as held, because, bus and timetable; its top-level S rules do not license a bare NP fragment or a multi-clause message joined by "and also"; and the wh-gap templates do not thread a gap through arbitrary embedding.

Longer input increases the number of spans and possible constituent combinations. A chart's number of span positions grows quadratically with token count, while general CFG parsing has cubic worst-case time in sentence length. Ambiguity can rise even faster in a particular grammar: with k PPs that each have two independent attachment sites, a simplified model has 2^k attachment combinations. Thus 5 such PPs allow 32 combinations and 10 allow 1,024 before lexical/feature constraints. These are model estimates, not measured counts for arbitrary helpdesk messages.

B. Evaluation of original design on new messages (2 Marks)

The following three messages were passed through the original grammar's lexical-coverage checker. All three fail immediately on an out-of-vocabulary token, so the CFG parser does not reach structural parsing. This is a direct measurement of lexical failure. The subsequent structural failures described in the last column are hand analyses of what remains unsupported even after a coverage repair.

ID and message	First OOV / all OOV	Original result	What still fails after vocabulary repair
D1: Hi, can you tell me when the exams that were postponed will be held, and also whether the hostel fee is due?	hi; held; also	coverage error: first OOV=hi	Multi-clause coordination with "and also whether"; passive "will be held"; embedded question plus relative clause.
D2: Hey, I missed the exam because my bus was delayed. Can you tell me the new timetable?	hey; missed; because; bus; delayed; timetable	coverage error: first OOV=hey	Causal subordinate clause "because …"; clause boundary coordination; several unknown content words and passive morphology.
D3: fees? hostel too, please check my payment.	too; please	coverage error: first OOV=too	"fees?" and "hostel too" are fragments; "please" politeness marker; no complete root S for the first NP fragment.

The first OOV tokens are hi, hey and too. The first example also contains held and also outside the original lexicon; the second contains missed, because, bus, delayed and timetable; the third contains please. The NLTK parser reports a coverage error before it returns parse counts. After those words are added, the original rule inventory still lacks the required multi-clause message structure, causal subordination and fragment roots. The example explicitly asked in the assignment contains a relative clause ("that were postponed"), an embedded wh clause ("when …"), passive/periphrastic verbal material ("will be held"), and coordinated interrogative clauses. The current grammar has a simple relative pattern

and an embedded wh pattern, but not their full nested combination with passive auxiliary chains and coordination.

The 20-token controlled sentence generated 92 CFG parses and 480 chart edges. Its chart times were 9.52 ms top-down and 9.16 ms Earley, while the current Viterbi implementation took 136.81 ms. This indicates that length alone need not make Earley impractical on this grammar, but attachment ambiguity makes output and ranking cost grow. It does not predict production latency for unrestricted user messages.

C. Adaptation / redesign (2 Marks)

The revised pipeline has four stages. First, normalize punctuation and conversational markers without deleting potentially meaningful content; preserve the original text for display. Second, run a feature-based Earley parser with rules for coordination, embedded declaratives and questions, passive auxiliary chains, causal clauses, and fragment roots. Third, use GAP features for wh-extraction and relative clauses so a missing NP/PP slot can be passed through embedded structure and unified at the filler. Fourth, use a larger in-domain PCFG or neural ranking signal for ambiguity, with a robust partial-parse fallback when the full sentence has no complete parse. The parser returns both the best analysis and an uncertainty/coverage indicator.

Grammar / parser change	Expected effect	Effort / risk
Add MSG → GREETING? CLAUSE (COORD CLAUSE)*; CLAUSE → S | FRAG; FRAG → NP | VP | WH	Accepts greetings, coordinated clauses and common fragments. Improves conversational coverage; adds attachment ambiguity.	Moderate grammar and regression-test effort; do not mistake a fragment for a complete request.
Add S/VP rules for passive auxiliaries and subordinators such as because, that and whether	Covers “were postponed,” “will be held,” and causal/embedded clauses.	Lexical and morphology expansion; auxiliary sequences must be constrained to avoid overgeneration.
Thread GAP features through WHCLAUSE and REL; unify case/role at the gap	Connects wh fillers and relative heads to the missing constituent slot.	Feature-aware Earley implementation is more complex; feature-state keys must preserve distinctions.
Increase annotated helpdesk treebank; estimate lexicalized/smoothed PCFG; recalibrate on held-out messages	Improves PP attachment and construction ranking.	Annotation becomes the main recurring cost; drift and rare constructions remain.
Fallback: partial chart, clause segmentation, intent/slot extraction, and abstention	Returns useful partial structure for OOV words, ellipsis and unparsable spans.	Requires confidence thresholds, audit logs and safe handling of uncertain intents.

Quantified effect. The original grammar’s measured input coverage on the three new examples is 0/3 because each has an OOV token. The modified system is designed to expose the uncovered span and preserve any parseable clause; a full-parse target of at least 3/3 on these same three is a regression-test goal, not a reported result from the current prototype. The sample rule additions are estimated at roughly 12–20 structural productions plus new lexical entries and feature constraints. This is an engineering estimate. The adaptation should reduce OOV-driven hard failures, but may increase ambiguity and chart states; the fallback keeps cost bounded by a maximum message length, beam/parse cap and time budget.

D. Evidence-based defence (2 Marks)

The measured evidence supports modification. On the narrow controlled set, the transparent CFG and chart parse most tested strings, reject missing required complements, and expose PP ambiguity. The same model rejects all three new conversational examples at lexical coverage before syntax is even considered. The longer test already shows 92 parses at 20 tokens, so adding coordination, subordinate clauses and wh-gap threading without ranking or pruning would worsen ambiguity. Conversely, the chart parser’s 9–10 ms result on that controlled 20-token sentence suggests the core need not be discarded. A broader treebank and PCFG ranking address ambiguity, while a partial parser prevents the expected 20% out-of-grammar traffic from becoming a total failure.

The revised design therefore retains Earley for transparent structural analysis, modifies grammar and feature handling for clause-level dependencies, and supplements it with probabilistic ranking and robust fallback. Its remaining limitations are explicit: the current experiment does not implement full feature-aware parsing, the three new messages are targeted diagnostics rather than a representative sample, and the PCFG treebank has only 20 hand-annotated trees. Production accuracy, fallback rate and latency must be measured on a de-identified, consented, independently annotated corpus before deployment.

Appendix A – Complete Structural Grammar and Lexicon

This generated inventory is the actual grammar used by the parser. It contains 53 structural productions and 179 distinct word forms grouped by lexical category. Lexical classes may overlap intentionally; for example, library can be a proper name or a common noun, and open can be an adjective, intransitive verb or transitive verb. Such ambiguity is controlled by the parse tree and should be refined with lexical features in a deployment grammar.

Phrase-structure rules: 53
Unique lexicon words: 179

S -> DECL
S -> IMP
S -> YNQ
S -> WHQ
DECL -> NP VP
IMP -> VP
YNQ -> AUX NP VP
WHQ -> WH NP VP
WHQ -> WH AUX NP VP
WHQ -> WH VP
WHQ -> WH COP NP
WHQ -> WH COP ADJP
WHQ -> WH AUX NP V_TRANS
NP -> DET NBAR
NP -> NBAR
NP -> PRON
NP -> PN
NP -> NP PP
NP -> NP REL
NP -> NP CONJ NP
NBAR -> N
NBAR -> ADJ NBAR
NBAR -> N NBAR
NBAR -> NUM N
NBAR -> NBAR CONJ NBAR
VP -> V_INTR
VP -> V_TRANS NP
VP -> V_DITR NP NP
VP -> V_OBJCOMP NP S
VP -> V_OBJCOMP NP THAT S
VP -> V_SENT THAT S
VP -> V_WH NP WHCLAUSE
VP -> V_PP PP
VP -> COP ADJP
VP -> COP NP
VP -> AUX VP
VP -> AUX V_INTR
VP -> VP PP
VP -> VP CONJ VP
VP -> V_TRANS NP PP
VP -> V_DITR NP PP
VP -> V_INTR PP
VP -> V_INTR CONJ VP
PP -> PREP NP
REL -> RELPRO VP
REL -> RELPRO AUX V_INTR
REL -> RELPRO AUX VP
WHCLAUSE -> WH NP VP
WHCLAUSE -> WH VP
WHCLAUSE -> WHETHER S
ADJP -> ADJ
ADJP -> ADJ PP
ADJP -> ADJP CONJ ADJP

LEXICON BY CATEGORY
DET: a, an, each, my, our, that, the, these, this, those, your

PRON: he, her, him, i, it, me, she, them, they, us, we, you
PN: arun, hostel, library, maya
N: account, application, book, campus, charge, charges, class, course, deadline, document, exam, exams, fee, fees, form, forms, grade, hostel, id, ids, lab, late, library, map, noon, notice, notices, office, password, payment, placement, portal, receipt, result, room, rooms, schedule, scholarship, seminar, service, signal, staff, student, students, time
ADJ: available, campus, due, final, high, late, missing, new, online, open, postponed, slow
NUM: five, many, one, three, two
V_INTR: arrived, crashed, crashes, expire, expired, open, opened, opens, postponed, sleep, starts, wait, waits, work, works
V_TRANS: accept, accepts, attend, attends, book, books, check, checks, need, needs, open, opens, pay, pays, read, reads, saw, show, shows, submit, submits, submitted, update, updates
V_DITR: give, gives, send, sends, sent, show, shows, tell, tells
V_OBJCOMP: ask, asks, tell, tells
V_SENT: know, knows, say, says
V_WH: ask, asks, tell, tells
V_PP: applies, apply, wait, waits
AUX: are, can, could, did, do, does, has, have, is, may, might, should, was, were, will, would
COP: are, be, is, seem, seems, was, were
PREP: about, after, at, before, by, during, for, from, in, near, of, on, to, under, with
CONJ: and, but, or
WH: how, what, when, where, who, why
WHETHER: whether
RELPRO: that, which, who
THAT: that

Appendix B – Top-Down Chart Traces and Ambiguous Trees

The first two trace listings are generated from NLTK's top-down chart parser. Edges are printed in parser order. They are implementation traces, not hand-invented recursive-descent stacks. The ambiguity examples preserve multiple tree structures for the same terminal string.

```
Unambiguous trace: the portal crashed
Edges=101
[0:1] 'the'
[1:2] 'portal'
[2:3] 'crashed'
[0:0] S  -> * DECL
[0:0] S  -> * IMP
[0:0] S  -> * YNQ
[0:0] S  -> * WHQ
[0:0] DECL -> * NP VP
[0:0] NP -> * DET NBAR
[0:0] NP -> * NBAR
[0:0] NP -> * PRON
[0:0] NP -> * PN
[0:0] NP -> * NP PP
[0:0] NP -> * NP REL
[0:0] NP -> * NP CONJ NP
[0:0] NBAR -> * N
[0:0] NBAR -> * ADJ NBAR
[0:0] NBAR -> * N NBAR
[0:0] NBAR -> * NUM N
[0:0] NBAR -> * NBAR CONJ NBAR
[0:0] DET -> * 'the'
[0:1] DET -> 'the' *
[0:1] NP -> DET * NBAR
[1:1] NBAR -> * N
[1:1] NBAR -> * ADJ NBAR
[1:1] NBAR -> * N NBAR
[1:1] NBAR -> * NUM N
[1:1] NBAR -> * NBAR CONJ NBAR
[1:1] N  -> * 'portal'
[1:2] N  -> 'portal' *
[1:2] NBAR -> N *
[1:2] NBAR -> N * NBAR
[2:2] NBAR -> * N
[2:2] NBAR -> * ADJ NBAR
[2:2] NBAR -> * N NBAR
[2:2] NBAR -> * NUM N
[2:2] NBAR -> * NBAR CONJ NBAR
[0:2] NP -> DET NBAR *
[1:2] NBAR -> NBAR * CONJ NBAR
[0:2] DECL -> NP * VP
[0:2] NP -> NP * PP
[0:2] NP -> NP * REL
[0:2] NP -> NP * CONJ NP
[2:2] REL -> * RELPRO VP
[2:2] REL -> * RELPRO AUX V_INTR
[2:2] REL -> * RELPRO AUX VP
[2:2] PP -> * PREP NP
[2:2] VP -> * V_INTR
[2:2] VP -> * V_TRANS NP
[2:2] VP -> * V_DITR NP NP
[2:2] VP -> * V_OBJCOMP NP S
[2:2] VP -> * V_OBJCOMP NP THAT S
[2:2] VP -> * V_SENT THAT S
[2:2] VP -> * V_WH NP WHCLAUSE
[2:2] VP -> * V_PP PP
[2:2] VP -> * COP ADJP
[2:2] VP -> * COP NP
[2:2] VP -> * AUX VP
[2:2] VP -> * AUX V_INTR
[2:2] VP -> * VP PP
[2:2] VP -> * VP CONJ VP
[2:2] VP -> * V_TRANS NP PP
[2:2] VP -> * V_DITR NP PP
[2:2] VP -> * V_INTR PP
```

[2:2] VP -> * V_INTR CONJ VP
[2:2] V_INTR -> * 'crashed'
[2:3] V_INTR -> 'crashed' *
[2:3] VP -> V_INTR *
[2:3] VP -> V_INTR * PP
[2:3] VP -> V_INTR * CONJ VP
[3:3] PP -> * PREP NP
[0:3] DECL -> NP VP *
[2:3] VP -> VP * PP
[2:3] VP -> VP * CONJ VP
[0:3] S -> DECL *
[0:0] IMP -> * VP
[0:0] VP -> * V_INTR
[0:0] VP -> * V_TRANS NP
[0:0] VP -> * V_DITR NP NP
[0:0] VP -> * V_OBJCOMP NP S
[0:0] VP -> * V_OBJCOMP NP THAT S
[0:0] VP -> * V_SENT THAT S
[0:0] VP -> * V_WH NP WHCLAUSE
[0:0] VP -> * V_PP PP
[0:0] VP -> * COP ADJP
[0:0] VP -> * COP NP
[0:0] VP -> * AUX VP
[0:0] VP -> * AUX V_INTR
[0:0] VP -> * VP PP
[0:0] VP -> * VP CONJ VP
[0:0] VP -> * V_TRANS NP PP
[0:0] VP -> * V_DITR NP PP
[0:0] VP -> * V_INTR PP
[0:0] VP -> * V_INTR CONJ VP
[0:0] YNQ -> * AUX NP VP
[0:0] WHQ -> * WH NP VP
[0:0] WHQ -> * WH AUX NP VP
[0:0] WHQ -> * WH VP
[0:0] WHQ -> * WH COP NP
[0:0] WHQ -> * WH COP ADJP
[0:0] WHQ -> * WH AUX NP V_TRANS

Ambiguous PP attachment trace: show the students with the late fee notice
Edges=283
[0:1] 'show'
[1:2] 'the'
[2:3] 'students'
[3:4] 'with'
[4:5] 'the'
[5:6] 'late'
[6:7] 'fee'
[7:8] 'notice'
[0:0] S -> * DECL
[0:0] S -> * IMP
[0:0] S -> * YNQ
[0:0] S -> * WHQ
[0:0] DECL -> * NP VP
[0:0] NP -> * DET NBAR
[0:0] NP -> * NBAR
[0:0] NP -> * PRON
[0:0] NP -> * PN
[0:0] NP -> * NP PP
[0:0] NP -> * NP REL
[0:0] NP -> * NP CONJ NP
[0:0] NBAR -> * N
[0:0] NBAR -> * ADJ NBAR
[0:0] NBAR -> * N NBAR
[0:0] NBAR -> * NUM N
[0:0] NBAR -> * NBAR CONJ NBAR
[0:0] IMP -> * VP
[0:0] VP -> * V_INTR
[0:0] VP -> * V_TRANS NP
[0:0] VP -> * V_DITR NP NP
[0:0] VP -> * V_OBJCOMP NP S
[0:0] VP -> * V_OBJCOMP NP THAT S
[0:0] VP -> * V_SENT THAT S
[0:0] VP -> * V_WH NP WHCLAUSE

```
[0:0] VP -> * V_PP PP
[0:0] VP -> * COP ADJP
[0:0] VP -> * COP NP
[0:0] VP -> * AUX VP
[0:0] VP -> * AUX V_INTR
[0:0] VP -> * VP PP
[0:0] VP -> * VP CONJ VP
[0:0] VP -> * V_TRANS NP PP
[0:0] VP -> * V_DITR NP PP
[0:0] VP -> * V_INTR PP
[0:0] VP -> * V_INTR CONJ VP
[0:0] V_DITR -> * 'show'
[0:1] V_DITR -> 'show' *
[0:1] VP -> V_DITR * NP NP
[0:1] VP -> V_DITR * NP PP
[1:1] NP -> * DET NBAR
[1:1] NP -> * NBAR
[1:1] NP -> * PRON
[1:1] NP -> * PN
[1:1] NP -> * NP PP
[1:1] NP -> * NP REL
[1:1] NP -> * NP CONJ NP
[1:1] NBAR -> * N
[1:1] NBAR -> * ADJ NBAR
[1:1] NBAR -> * N NBAR
[1:1] NBAR -> * NUM N
[1:1] NBAR -> * NBAR CONJ NBAR
[1:1] DET -> * 'the'
[1:2] DET -> 'the' *
[1:2] NP -> DET * NBAR
[2:2] NBAR -> * N
[2:2] NBAR -> * ADJ NBAR
[2:2] NBAR -> * N NBAR
[2:2] NBAR -> * NUM N
[2:2] NBAR -> * NBAR CONJ NBAR
[2:2] N  -> * 'students'
[2:3] N  -> 'students' *
[2:3] NBAR -> N *
[2:3] NBAR -> N * NBAR
[3:3] NBAR -> * N
[3:3] NBAR -> * ADJ NBAR
[3:3] NBAR -> * N NBAR
[3:3] NBAR -> * NUM N
[3:3] NBAR -> * NBAR CONJ NBAR
[1:3] NP -> DET NBAR *
[2:3] NBAR -> NBAR * CONJ NBAR
[0:3] VP -> V_DITR NP * NP
[0:3] VP -> V_DITR NP * PP
[1:3] NP -> NP * PP
[1:3] NP -> NP * REL
[1:3] NP -> NP * CONJ NP
[3:3] REL -> * RELPRO VP
[3:3] REL -> * RELPRO AUX V_INTR
[3:3] REL -> * RELPRO AUX VP
[3:3] PP -> * PREP NP
[3:3] PREP -> * 'with'
[3:4] PREP -> 'with' *
[3:4] PP -> PREP * NP
[4:4] NP -> * DET NBAR
[4:4] NP -> * NBAR
[4:4] NP -> * PRON
[4:4] NP -> * PN
[4:4] NP -> * NP PP
[4:4] NP -> * NP REL
[4:4] NP -> * NP CONJ NP
[4:4] NBAR -> * N
[4:4] NBAR -> * ADJ NBAR
[4:4] NBAR -> * N NBAR
[4:4] NBAR -> * NUM N
[4:4] NBAR -> * NBAR CONJ NBAR
[4:4] DET -> * 'the'
[4:5] DET -> 'the' *
[4:5] NP -> DET * NBAR
```

```
[5:5] NBAR -> * N
[5:5] NBAR -> * ADJ NBAR
[5:5] NBAR -> * N NBAR
[5:5] NBAR -> * NUM N
[5:5] NBAR -> * NBAR CONJ NBAR
[5:5] N  -> * 'late'
[5:6] N  -> 'late' *
[5:6] NBAR -> N *
[5:6] NBAR -> N * NBAR
[6:6] NBAR -> * N
[6:6] NBAR -> * ADJ NBAR
[6:6] NBAR -> * N NBAR
[6:6] NBAR -> * NUM N
[6:6] NBAR -> * NBAR CONJ NBAR
```

Competing parse trees

```
show the students with the late fee notice
Showing 3 of the available parses.
Parse 1: (S
  (IMP
    (VP
      (V_DITR show)
      (NP (NP (DET the) (NBAR (N students))) (PP (PREP with) (NP (DET the) (NBAR (N late) (NBAR (N fee))))))
      (NP (NBAR (N notice))))))
Parse 2: (S
  (IMP
    (VP
      (V_DITR show)
      (NP (NP (DET the) (NBAR (N students))) (PP (PREP with) (NP (DET the) (NBAR (ADJ late) (NBAR (N fee))))))
      (NP (NBAR (N notice))))))
Parse 3: (S
  (IMP
    (VP
      (V_DITR show)
      (NP (NP (DET the) (NBAR (N students))) (PP (PREP with) (NP (DET the) (NBAR (N late)))))
      (NP (NBAR (N fee) (NBAR (N notice)))))))


book the room near the library for the seminar
Showing 3 of the available parses.
Parse 1: (S
  (IMP
    (VP
      (V_TRANS book)
      (NP
        (NP (NP (DET the) (NBAR (N room))) (PP (PREP near) (NP (DET the) (NBAR (N library)))))
        (PP (PREP for) (NP (DET the) (NBAR (N seminar))))))))
Parse 2: (S
  (IMP
    (VP
      (V_TRANS book)
      (NP
        (NP (DET the) (NBAR (N room)))
        (PP (PREP near) (NP (NP (DET the) (NBAR (N library))) (PP (PREP for) (NP (DET the) (NBAR (N seminar)))))))))
Parse 3: (S
  (IMP
    (VP
      (V_TRANS book)
      (NP (NP (DET the) (NBAR (N room))) (PP (PREP near) (NP (DET the) (NBAR (N library)))))
      (PP (PREP for) (NP (DET the) (NBAR (N seminar)))))))


i saw the notice with a map
Showing 3 of the available parses.
Parse 1: (S (DECL (NP (PRON i)) (VP (VP (V_TRANS saw) (NP (DET the) (NBAR (N notice)))) (PP (PREP with) (NP (DET a) (NBAR
(N map)))))))
Parse 2: (S (DECL (NP (PRON i)) (VP (V_TRANS saw) (NP (NP (DET the) (NBAR (N notice))) (PP (PREP with) (NP (DET a) (NBAR
(N map))))))))
Parse 3: (S (DECL (NP (PRON i)) (VP (V_TRANS saw) (NP (DET the) (NBAR (N notice))) (PP (PREP with) (NP (DET a) (NBAR (N
map)))))))
```

Appendix C – Complete Earley Chart with State Origins and Operations

Every item below is printed as [LHS → prefix • suffix, origin], with the chart column shown by S[i]. The final bracket records the operation that first introduced the state. The state source is unique because duplicate item keys are ignored. No ε-rules occur in this run.

```
Sentence: the student submits the late form
Tokens: ['the', 'student', 'submits', 'the', 'late', 'form']
Accepted: True


S[0] (226 states)
  [ROOT -> • S, origin=0]  [initialize]
  [S -> • DECL, origin=0]  [PREDICTOR from ROOT]
  [S -> • IMP, origin=0]  [PREDICTOR from ROOT]
  [S -> • YNQ, origin=0]  [PREDICTOR from ROOT]
  [S -> • WHQ, origin=0]  [PREDICTOR from ROOT]
  [DECL -> • NP VP, origin=0]  [PREDICTOR from S]
  [IMP -> • VP, origin=0]  [PREDICTOR from S]
  [YNQ -> • AUX NP VP, origin=0]  [PREDICTOR from S]
  [WHQ -> • WH NP VP, origin=0]  [PREDICTOR from S]
  [WHQ -> • WH AUX NP VP, origin=0]  [PREDICTOR from S]
  [WHQ -> • WH VP, origin=0]  [PREDICTOR from S]
  [WHQ -> • WH COP NP, origin=0]  [PREDICTOR from S]
  [WHQ -> • WH COP ADJP, origin=0]  [PREDICTOR from S]
  [WHQ -> • WH AUX NP V_TRANS, origin=0]  [PREDICTOR from S]
  [NP -> • DET NBAR, origin=0]  [PREDICTOR from DECL]
  [NP -> • NBAR, origin=0]  [PREDICTOR from DECL]
  [NP -> • PRON, origin=0]  [PREDICTOR from DECL]
  [NP -> • PN, origin=0]  [PREDICTOR from DECL]
  [NP -> • NP PP, origin=0]  [PREDICTOR from DECL]
  [NP -> • NP REL, origin=0]  [PREDICTOR from DECL]
  [NP -> • NP CONJ NP, origin=0]  [PREDICTOR from DECL]
  [VP -> • V_INTR, origin=0]  [PREDICTOR from IMP]
  [VP -> • V_TRANS NP, origin=0]  [PREDICTOR from IMP]
  [VP -> • V_DITR NP NP, origin=0]  [PREDICTOR from IMP]
  [VP -> • V_OBJCOMP NP S, origin=0]  [PREDICTOR from IMP]
  [VP -> • V_OBJCOMP NP THAT S, origin=0]  [PREDICTOR from IMP]
  [VP -> • V_SENT THAT S, origin=0]  [PREDICTOR from IMP]
  [VP -> • V_WH NP WHCLAUSE, origin=0]  [PREDICTOR from IMP]
  [VP -> • V_PP PP, origin=0]  [PREDICTOR from IMP]
  [VP -> • COP ADJP, origin=0]  [PREDICTOR from IMP]
  [VP -> • COP NP, origin=0]  [PREDICTOR from IMP]
  [VP -> • AUX VP, origin=0]  [PREDICTOR from IMP]
  [VP -> • AUX V_INTR, origin=0]  [PREDICTOR from IMP]
  [VP -> • VP PP, origin=0]  [PREDICTOR from IMP]
  [VP -> • VP CONJ VP, origin=0]  [PREDICTOR from IMP]
  [VP -> • V_TRANS NP PP, origin=0]  [PREDICTOR from IMP]
  [VP -> • V_DITR NP PP, origin=0]  [PREDICTOR from IMP]
  [VP -> • V_INTR PP, origin=0]  [PREDICTOR from IMP]
  [VP -> • V_INTR CONJ VP, origin=0]  [PREDICTOR from IMP]
  [AUX -> • are, origin=0]  [PREDICTOR from YNQ]
  [AUX -> • can, origin=0]  [PREDICTOR from YNQ]
  [AUX -> • could, origin=0]  [PREDICTOR from YNQ]
  [AUX -> • did, origin=0]  [PREDICTOR from YNQ]
  [AUX -> • do, origin=0]  [PREDICTOR from YNQ]
  [AUX -> • does, origin=0]  [PREDICTOR from YNQ]
  [AUX -> • has, origin=0]  [PREDICTOR from YNQ]
  [AUX -> • have, origin=0]  [PREDICTOR from YNQ]
  [AUX -> • is, origin=0]  [PREDICTOR from YNQ]
  [AUX -> • may, origin=0]  [PREDICTOR from YNQ]
  [AUX -> • might, origin=0]  [PREDICTOR from YNQ]
  [AUX -> • should, origin=0]  [PREDICTOR from YNQ]
  [AUX -> • was, origin=0]  [PREDICTOR from YNQ]
  [AUX -> • were, origin=0]  [PREDICTOR from YNQ]
  [AUX -> • will, origin=0]  [PREDICTOR from YNQ]
  [AUX -> • would, origin=0]  [PREDICTOR from YNQ]
  [WH -> • how, origin=0]  [PREDICTOR from WHQ]
  [WH -> • what, origin=0]  [PREDICTOR from WHQ]
  [WH -> • when, origin=0]  [PREDICTOR from WHQ]
  [WH -> • where, origin=0]  [PREDICTOR from WHQ]
  [WH -> • who, origin=0]  [PREDICTOR from WHQ]
  [WH -> • why, origin=0]  [PREDICTOR from WHQ]
  [DET -> • a, origin=0]  [PREDICTOR from NP]
  [DET -> • an, origin=0]  [PREDICTOR from NP]
  [DET -> • each, origin=0]  [PREDICTOR from NP]
  [DET -> • my, origin=0]  [PREDICTOR from NP]
```

[DET -> • our, origin=0] [PREDICTOR from NP]
[DET -> • that, origin=0] [PREDICTOR from NP]
[DET -> • the, origin=0] [PREDICTOR from NP]
[DET -> • these, origin=0] [PREDICTOR from NP]
[DET -> • this, origin=0] [PREDICTOR from NP]
[DET -> • those, origin=0] [PREDICTOR from NP]
[DET -> • your, origin=0] [PREDICTOR from NP]
[NBAR -> • N, origin=0] [PREDICTOR from NP]
[NBAR -> • ADJ NBAR, origin=0] [PREDICTOR from NP]
[NBAR -> • N NBAR, origin=0] [PREDICTOR from NP]
[NBAR -> • NUM N, origin=0] [PREDICTOR from NP]
[NBAR -> • NBAR CONJ NBAR, origin=0] [PREDICTOR from NP]
[PRON -> • he, origin=0] [PREDICTOR from NP]
[PRON -> • her, origin=0] [PREDICTOR from NP]
[PRON -> • him, origin=0] [PREDICTOR from NP]
[PRON -> • i, origin=0] [PREDICTOR from NP]
[PRON -> • it, origin=0] [PREDICTOR from NP]
[PRON -> • me, origin=0] [PREDICTOR from NP]
[PRON -> • she, origin=0] [PREDICTOR from NP]
[PRON -> • them, origin=0] [PREDICTOR from NP]
[PRON -> • they, origin=0] [PREDICTOR from NP]
[PRON -> • us, origin=0] [PREDICTOR from NP]
[PRON -> • we, origin=0] [PREDICTOR from NP]
[PRON -> • you, origin=0] [PREDICTOR from NP]
[PN -> • arun, origin=0] [PREDICTOR from NP]
[PN -> • hostel, origin=0] [PREDICTOR from NP]
[PN -> • library, origin=0] [PREDICTOR from NP]
[PN -> • maya, origin=0] [PREDICTOR from NP]
[V_INTR -> • arrived, origin=0] [PREDICTOR from VP]
[V_INTR -> • crashed, origin=0] [PREDICTOR from VP]
[V_INTR -> • crashes, origin=0] [PREDICTOR from VP]
[V_INTR -> • expire, origin=0] [PREDICTOR from VP]
[V_INTR -> • expired, origin=0] [PREDICTOR from VP]
[V_INTR -> • open, origin=0] [PREDICTOR from VP]
[V_INTR -> • opened, origin=0] [PREDICTOR from VP]
[V_INTR -> • opens, origin=0] [PREDICTOR from VP]
[V_INTR -> • postponed, origin=0] [PREDICTOR from VP]
[V_INTR -> • sleep, origin=0] [PREDICTOR from VP]
[V_INTR -> • starts, origin=0] [PREDICTOR from VP]
[V_INTR -> • wait, origin=0] [PREDICTOR from VP]
[V_INTR -> • waits, origin=0] [PREDICTOR from VP]
[V_INTR -> • work, origin=0] [PREDICTOR from VP]
[V_INTR -> • works, origin=0] [PREDICTOR from VP]
[V_TRANS -> • accept, origin=0] [PREDICTOR from VP]
[V_TRANS -> • accepts, origin=0] [PREDICTOR from VP]
[V_TRANS -> • attend, origin=0] [PREDICTOR from VP]
[V_TRANS -> • attends, origin=0] [PREDICTOR from VP]
[V_TRANS -> • book, origin=0] [PREDICTOR from VP]
[V_TRANS -> • books, origin=0] [PREDICTOR from VP]
[V_TRANS -> • check, origin=0] [PREDICTOR from VP]
[V_TRANS -> • checks, origin=0] [PREDICTOR from VP]
[V_TRANS -> • need, origin=0] [PREDICTOR from VP]
[V_TRANS -> • needs, origin=0] [PREDICTOR from VP]
[V_TRANS -> • open, origin=0] [PREDICTOR from VP]
[V_TRANS -> • opens, origin=0] [PREDICTOR from VP]
[V_TRANS -> • pay, origin=0] [PREDICTOR from VP]
[V_TRANS -> • pays, origin=0] [PREDICTOR from VP]
[V_TRANS -> • read, origin=0] [PREDICTOR from VP]
[V_TRANS -> • reads, origin=0] [PREDICTOR from VP]
[V_TRANS -> • saw, origin=0] [PREDICTOR from VP]
[V_TRANS -> • show, origin=0] [PREDICTOR from VP]
[V_TRANS -> • shows, origin=0] [PREDICTOR from VP]
[V_TRANS -> • submit, origin=0] [PREDICTOR from VP]
[V_TRANS -> • submits, origin=0] [PREDICTOR from VP]
[V_TRANS -> • submitted, origin=0] [PREDICTOR from VP]
[V_TRANS -> • update, origin=0] [PREDICTOR from VP]
[V_TRANS -> • updates, origin=0] [PREDICTOR from VP]
[V_DITR -> • give, origin=0] [PREDICTOR from VP]
[V_DITR -> • gives, origin=0] [PREDICTOR from VP]
[V_DITR -> • send, origin=0] [PREDICTOR from VP]
[V_DITR -> • sends, origin=0] [PREDICTOR from VP]
[V_DITR -> • sent, origin=0] [PREDICTOR from VP]
[V_DITR -> • show, origin=0] [PREDICTOR from VP]
[V_DITR -> • shows, origin=0] [PREDICTOR from VP]
[V_DITR -> • tell, origin=0] [PREDICTOR from VP]
[V_DITR -> • tells, origin=0] [PREDICTOR from VP]
[V_OBJCOMP -> • ask, origin=0] [PREDICTOR from VP]
[V_OBJCOMP -> • asks, origin=0] [PREDICTOR from VP]
[V_OBJCOMP -> • tell, origin=0] [PREDICTOR from VP]

[V_OBJCOMP -> • tells, origin=0] [PREDICTOR from VP]
[V_SENT -> • know, origin=0] [PREDICTOR from VP]
[V_SENT -> • knows, origin=0] [PREDICTOR from VP]
[V_SENT -> • say, origin=0] [PREDICTOR from VP]
[V_SENT -> • says, origin=0] [PREDICTOR from VP]
[V_WH -> • ask, origin=0] [PREDICTOR from VP]
[V_WH -> • asks, origin=0] [PREDICTOR from VP]
[V_WH -> • tell, origin=0] [PREDICTOR from VP]
[V_WH -> • tells, origin=0] [PREDICTOR from VP]
[V_PP -> • applies, origin=0] [PREDICTOR from VP]
[V_PP -> • apply, origin=0] [PREDICTOR from VP]
[V_PP -> • wait, origin=0] [PREDICTOR from VP]
[V_PP -> • waits, origin=0] [PREDICTOR from VP]
[COP -> • are, origin=0] [PREDICTOR from VP]
[COP -> • be, origin=0] [PREDICTOR from VP]
[COP -> • is, origin=0] [PREDICTOR from VP]
[COP -> • seem, origin=0] [PREDICTOR from VP]
[COP -> • seems, origin=0] [PREDICTOR from VP]
[COP -> • was, origin=0] [PREDICTOR from VP]
[COP -> • were, origin=0] [PREDICTOR from VP]
[N -> • account, origin=0] [PREDICTOR from NBAR]
[N -> • application, origin=0] [PREDICTOR from NBAR]
[N -> • book, origin=0] [PREDICTOR from NBAR]
[N -> • campus, origin=0] [PREDICTOR from NBAR]
[N -> • charge, origin=0] [PREDICTOR from NBAR]
[N -> • charges, origin=0] [PREDICTOR from NBAR]
[N -> • class, origin=0] [PREDICTOR from NBAR]
[N -> • course, origin=0] [PREDICTOR from NBAR]
[N -> • deadline, origin=0] [PREDICTOR from NBAR]
[N -> • document, origin=0] [PREDICTOR from NBAR]
[N -> • exam, origin=0] [PREDICTOR from NBAR]
[N -> • exams, origin=0] [PREDICTOR from NBAR]
[N -> • fee, origin=0] [PREDICTOR from NBAR]
[N -> • fees, origin=0] [PREDICTOR from NBAR]
[N -> • form, origin=0] [PREDICTOR from NBAR]
[N -> • forms, origin=0] [PREDICTOR from NBAR]
[N -> • grade, origin=0] [PREDICTOR from NBAR]
[N -> • hostel, origin=0] [PREDICTOR from NBAR]
[N -> • id, origin=0] [PREDICTOR from NBAR]
[N -> • ids, origin=0] [PREDICTOR from NBAR]
[N -> • lab, origin=0] [PREDICTOR from NBAR]
[N -> • late, origin=0] [PREDICTOR from NBAR]
[N -> • library, origin=0] [PREDICTOR from NBAR]
[N -> • map, origin=0] [PREDICTOR from NBAR]
[N -> • noon, origin=0] [PREDICTOR from NBAR]
[N -> • notice, origin=0] [PREDICTOR from NBAR]
[N -> • notices, origin=0] [PREDICTOR from NBAR]
[N -> • office, origin=0] [PREDICTOR from NBAR]
[N -> • password, origin=0] [PREDICTOR from NBAR]
[N -> • payment, origin=0] [PREDICTOR from NBAR]
[N -> • placement, origin=0] [PREDICTOR from NBAR]
[N -> • portal, origin=0] [PREDICTOR from NBAR]
[N -> • receipt, origin=0] [PREDICTOR from NBAR]
[N -> • result, origin=0] [PREDICTOR from NBAR]
[N -> • room, origin=0] [PREDICTOR from NBAR]
[N -> • rooms, origin=0] [PREDICTOR from NBAR]
[N -> • schedule, origin=0] [PREDICTOR from NBAR]
[N -> • scholarship, origin=0] [PREDICTOR from NBAR]
[N -> • seminar, origin=0] [PREDICTOR from NBAR]
[N -> • service, origin=0] [PREDICTOR from NBAR]
[N -> • signal, origin=0] [PREDICTOR from NBAR]
[N -> • staff, origin=0] [PREDICTOR from NBAR]
[N -> • student, origin=0] [PREDICTOR from NBAR]
[N -> • students, origin=0] [PREDICTOR from NBAR]
[N -> • time, origin=0] [PREDICTOR from NBAR]
[ADJ -> • available, origin=0] [PREDICTOR from NBAR]
[ADJ -> • campus, origin=0] [PREDICTOR from NBAR]
[ADJ -> • due, origin=0] [PREDICTOR from NBAR]
[ADJ -> • final, origin=0] [PREDICTOR from NBAR]
[ADJ -> • high, origin=0] [PREDICTOR from NBAR]
[ADJ -> • late, origin=0] [PREDICTOR from NBAR]
[ADJ -> • missing, origin=0] [PREDICTOR from NBAR]
[ADJ -> • new, origin=0] [PREDICTOR from NBAR]
[ADJ -> • online, origin=0] [PREDICTOR from NBAR]
[ADJ -> • open, origin=0] [PREDICTOR from NBAR]
[ADJ -> • postponed, origin=0] [PREDICTOR from NBAR]
[ADJ -> • slow, origin=0] [PREDICTOR from NBAR]
[NUM -> • five, origin=0] [PREDICTOR from NBAR]
[NUM -> • many, origin=0] [PREDICTOR from NBAR]

[NUM -> • one, origin=0] [PREDICTOR from NBAR]
[NUM -> • three, origin=0] [PREDICTOR from NBAR]
[NUM -> • two, origin=0] [PREDICTOR from NBAR]
S[1] (69 states)
[DET -> the •, origin=0] [SCANNER the]
[NP -> DET • NBAR, origin=0] [COMPLETER DET]
[NBAR -> • N, origin=1] [PREDICTOR from NP]
[NBAR -> • ADJ NBAR, origin=1] [PREDICTOR from NP]
[NBAR -> • N NBAR, origin=1] [PREDICTOR from NP]
[NBAR -> • NUM N, origin=1] [PREDICTOR from NP]
[NBAR -> • NBAR CONJ NBAR, origin=1] [PREDICTOR from NP]
[N -> • account, origin=1] [PREDICTOR from NBAR]
[N -> • application, origin=1] [PREDICTOR from NBAR]
[N -> • book, origin=1] [PREDICTOR from NBAR]
[N -> • campus, origin=1] [PREDICTOR from NBAR]
[N -> • charge, origin=1] [PREDICTOR from NBAR]
[N -> • charges, origin=1] [PREDICTOR from NBAR]
[N -> • class, origin=1] [PREDICTOR from NBAR]
[N -> • course, origin=1] [PREDICTOR from NBAR]
[N -> • deadline, origin=1] [PREDICTOR from NBAR]
[N -> • document, origin=1] [PREDICTOR from NBAR]
[N -> • exam, origin=1] [PREDICTOR from NBAR]
[N -> • exams, origin=1] [PREDICTOR from NBAR]
[N -> • fee, origin=1] [PREDICTOR from NBAR]
[N -> • fees, origin=1] [PREDICTOR from NBAR]
[N -> • form, origin=1] [PREDICTOR from NBAR]
[N -> • forms, origin=1] [PREDICTOR from NBAR]
[N -> • grade, origin=1] [PREDICTOR from NBAR]
[N -> • hostel, origin=1] [PREDICTOR from NBAR]
[N -> • id, origin=1] [PREDICTOR from NBAR]
[N -> • ids, origin=1] [PREDICTOR from NBAR]
[N -> • lab, origin=1] [PREDICTOR from NBAR]
[N -> • late, origin=1] [PREDICTOR from NBAR]
[N -> • library, origin=1] [PREDICTOR from NBAR]
[N -> • map, origin=1] [PREDICTOR from NBAR]
[N -> • noon, origin=1] [PREDICTOR from NBAR]
[N -> • notice, origin=1] [PREDICTOR from NBAR]
[N -> • notices, origin=1] [PREDICTOR from NBAR]
[N -> • office, origin=1] [PREDICTOR from NBAR]
[N -> • password, origin=1] [PREDICTOR from NBAR]
[N -> • payment, origin=1] [PREDICTOR from NBAR]
[N -> • placement, origin=1] [PREDICTOR from NBAR]
[N -> • portal, origin=1] [PREDICTOR from NBAR]
[N -> • receipt, origin=1] [PREDICTOR from NBAR]
[N -> • result, origin=1] [PREDICTOR from NBAR]
[N -> • room, origin=1] [PREDICTOR from NBAR]
[N -> • rooms, origin=1] [PREDICTOR from NBAR]
[N -> • schedule, origin=1] [PREDICTOR from NBAR]
[N -> • scholarship, origin=1] [PREDICTOR from NBAR]
[N -> • seminar, origin=1] [PREDICTOR from NBAR]
[N -> • service, origin=1] [PREDICTOR from NBAR]
[N -> • signal, origin=1] [PREDICTOR from NBAR]
[N -> • staff, origin=1] [PREDICTOR from NBAR]
[N -> • student, origin=1] [PREDICTOR from NBAR]
[N -> • students, origin=1] [PREDICTOR from NBAR]
[N -> • time, origin=1] [PREDICTOR from NBAR]
[ADJ -> • available, origin=1] [PREDICTOR from NBAR]
[ADJ -> • campus, origin=1] [PREDICTOR from NBAR]
[ADJ -> • due, origin=1] [PREDICTOR from NBAR]
[ADJ -> • final, origin=1] [PREDICTOR from NBAR]
[ADJ -> • high, origin=1] [PREDICTOR from NBAR]
[ADJ -> • late, origin=1] [PREDICTOR from NBAR]
[ADJ -> • missing, origin=1] [PREDICTOR from NBAR]
[ADJ -> • new, origin=1] [PREDICTOR from NBAR]
[ADJ -> • online, origin=1] [PREDICTOR from NBAR]
[ADJ -> • open, origin=1] [PREDICTOR from NBAR]
[ADJ -> • postponed, origin=1] [PREDICTOR from NBAR]
[ADJ -> • slow, origin=1] [PREDICTOR from NBAR]
[NUM -> • five, origin=1] [PREDICTOR from NBAR]
[NUM -> • many, origin=1] [PREDICTOR from NBAR]
[NUM -> • one, origin=1] [PREDICTOR from NBAR]
[NUM -> • three, origin=1] [PREDICTOR from NBAR]
[NUM -> • two, origin=1] [PREDICTOR from NBAR]
S[2] (206 states)
[N -> student •, origin=1] [SCANNER student]
[NBAR -> N •, origin=1] [COMPLETER N]
[NBAR -> N • NBAR, origin=1] [COMPLETER N]
[NP -> DET NBAR •, origin=0] [COMPLETER NBAR]
[NBAR -> NBAR • CONJ NBAR, origin=1] [COMPLETER NBAR]

[NBAR -> • N, origin=2] [PREDICTOR from NBAR]
[NBAR -> • ADJ NBAR, origin=2] [PREDICTOR from NBAR]
[NBAR -> • N NBAR, origin=2] [PREDICTOR from NBAR]
[NBAR -> • NUM N, origin=2] [PREDICTOR from NBAR]
[NBAR -> • NBAR CONJ NBAR, origin=2] [PREDICTOR from NBAR]
[DECL -> NP • VP, origin=0] [COMPLETER NP]
[NP -> NP • PP, origin=0] [COMPLETER NP]
[NP -> NP • REL, origin=0] [COMPLETER NP]
[NP -> NP • CONJ NP, origin=0] [COMPLETER NP]
[CONJ -> • and, origin=2] [PREDICTOR from NBAR]
[CONJ -> • but, origin=2] [PREDICTOR from NBAR]
[CONJ -> • or, origin=2] [PREDICTOR from NBAR]
[N -> • account, origin=2] [PREDICTOR from NBAR]
[N -> • application, origin=2] [PREDICTOR from NBAR]
[N -> • book, origin=2] [PREDICTOR from NBAR]
[N -> • campus, origin=2] [PREDICTOR from NBAR]
[N -> • charge, origin=2] [PREDICTOR from NBAR]
[N -> • charges, origin=2] [PREDICTOR from NBAR]
[N -> • class, origin=2] [PREDICTOR from NBAR]
[N -> • course, origin=2] [PREDICTOR from NBAR]
[N -> • deadline, origin=2] [PREDICTOR from NBAR]
[N -> • document, origin=2] [PREDICTOR from NBAR]
[N -> • exam, origin=2] [PREDICTOR from NBAR]
[N -> • exams, origin=2] [PREDICTOR from NBAR]
[N -> • fee, origin=2] [PREDICTOR from NBAR]
[N -> • fees, origin=2] [PREDICTOR from NBAR]
[N -> • form, origin=2] [PREDICTOR from NBAR]
[N -> • forms, origin=2] [PREDICTOR from NBAR]
[N -> • grade, origin=2] [PREDICTOR from NBAR]
[N -> • hostel, origin=2] [PREDICTOR from NBAR]
[N -> • id, origin=2] [PREDICTOR from NBAR]
[N -> • ids, origin=2] [PREDICTOR from NBAR]
[N -> • lab, origin=2] [PREDICTOR from NBAR]
[N -> • late, origin=2] [PREDICTOR from NBAR]
[N -> • library, origin=2] [PREDICTOR from NBAR]
[N -> • map, origin=2] [PREDICTOR from NBAR]
[N -> • noon, origin=2] [PREDICTOR from NBAR]
[N -> • notice, origin=2] [PREDICTOR from NBAR]
[N -> • notices, origin=2] [PREDICTOR from NBAR]
[N -> • office, origin=2] [PREDICTOR from NBAR]
[N -> • password, origin=2] [PREDICTOR from NBAR]
[N -> • payment, origin=2] [PREDICTOR from NBAR]
[N -> • placement, origin=2] [PREDICTOR from NBAR]
[N -> • portal, origin=2] [PREDICTOR from NBAR]
[N -> • receipt, origin=2] [PREDICTOR from NBAR]
[N -> • result, origin=2] [PREDICTOR from NBAR]
[N -> • room, origin=2] [PREDICTOR from NBAR]
[N -> • rooms, origin=2] [PREDICTOR from NBAR]
[N -> • schedule, origin=2] [PREDICTOR from NBAR]
[N -> • scholarship, origin=2] [PREDICTOR from NBAR]
[N -> • seminar, origin=2] [PREDICTOR from NBAR]
[N -> • service, origin=2] [PREDICTOR from NBAR]
[N -> • signal, origin=2] [PREDICTOR from NBAR]
[N -> • staff, origin=2] [PREDICTOR from NBAR]
[N -> • student, origin=2] [PREDICTOR from NBAR]
[N -> • students, origin=2] [PREDICTOR from NBAR]
[N -> • time, origin=2] [PREDICTOR from NBAR]
[ADJ -> • available, origin=2] [PREDICTOR from NBAR]
[ADJ -> • campus, origin=2] [PREDICTOR from NBAR]
[ADJ -> • due, origin=2] [PREDICTOR from NBAR]
[ADJ -> • final, origin=2] [PREDICTOR from NBAR]
[ADJ -> • high, origin=2] [PREDICTOR from NBAR]
[ADJ -> • late, origin=2] [PREDICTOR from NBAR]
[ADJ -> • missing, origin=2] [PREDICTOR from NBAR]
[ADJ -> • new, origin=2] [PREDICTOR from NBAR]
[ADJ -> • online, origin=2] [PREDICTOR from NBAR]
[ADJ -> • open, origin=2] [PREDICTOR from NBAR]
[ADJ -> • postponed, origin=2] [PREDICTOR from NBAR]
[ADJ -> • slow, origin=2] [PREDICTOR from NBAR]
[NUM -> • five, origin=2] [PREDICTOR from NBAR]
[NUM -> • many, origin=2] [PREDICTOR from NBAR]
[NUM -> • one, origin=2] [PREDICTOR from NBAR]
[NUM -> • three, origin=2] [PREDICTOR from NBAR]
[NUM -> • two, origin=2] [PREDICTOR from NBAR]
[VP -> • V_INTR, origin=2] [PREDICTOR from DECL]
[VP -> • V_TRANS NP, origin=2] [PREDICTOR from DECL]
[VP -> • V_DITR NP NP, origin=2] [PREDICTOR from DECL]
[VP -> • V_OBJCOMP NP S, origin=2] [PREDICTOR from DECL]
[VP -> • V_OBJCOMP NP THAT S, origin=2] [PREDICTOR from DECL]

[VP -> • V_SENT THAT S, origin=2] [PREDICTOR from DECL]
[VP -> • V_WH NP WHCLAUSE, origin=2] [PREDICTOR from DECL]
[VP -> • V_PP PP, origin=2] [PREDICTOR from DECL]
[VP -> • COP ADJP, origin=2] [PREDICTOR from DECL]
[VP -> • COP NP, origin=2] [PREDICTOR from DECL]
[VP -> • AUX VP, origin=2] [PREDICTOR from DECL]
[VP -> • AUX V_INTR, origin=2] [PREDICTOR from DECL]
[VP -> • VP PP, origin=2] [PREDICTOR from DECL]
[VP -> • VP CONJ VP, origin=2] [PREDICTOR from DECL]
[VP -> • V_TRANS NP PP, origin=2] [PREDICTOR from DECL]
[VP -> • V_DITR NP PP, origin=2] [PREDICTOR from DECL]
[VP -> • V_INTR PP, origin=2] [PREDICTOR from DECL]
[VP -> • V_INTR CONJ VP, origin=2] [PREDICTOR from DECL]
[PP -> • PREP NP, origin=2] [PREDICTOR from NP]
[REL -> • RELPRO VP, origin=2] [PREDICTOR from NP]
[REL -> • RELPRO AUX V_INTR, origin=2] [PREDICTOR from NP]
[REL -> • RELPRO AUX VP, origin=2] [PREDICTOR from NP]
[V_INTR -> • arrived, origin=2] [PREDICTOR from VP]
[V_INTR -> • crashed, origin=2] [PREDICTOR from VP]
[V_INTR -> • crashes, origin=2] [PREDICTOR from VP]
[V_INTR -> • expire, origin=2] [PREDICTOR from VP]
[V_INTR -> • expired, origin=2] [PREDICTOR from VP]
[V_INTR -> • open, origin=2] [PREDICTOR from VP]
[V_INTR -> • opened, origin=2] [PREDICTOR from VP]
[V_INTR -> • opens, origin=2] [PREDICTOR from VP]
[V_INTR -> • postponed, origin=2] [PREDICTOR from VP]
[V_INTR -> • sleep, origin=2] [PREDICTOR from VP]
[V_INTR -> • starts, origin=2] [PREDICTOR from VP]
[V_INTR -> • wait, origin=2] [PREDICTOR from VP]
[V_INTR -> • waits, origin=2] [PREDICTOR from VP]
[V_INTR -> • work, origin=2] [PREDICTOR from VP]
[V_INTR -> • works, origin=2] [PREDICTOR from VP]
[V_TRANS -> • accept, origin=2] [PREDICTOR from VP]
[V_TRANS -> • accepts, origin=2] [PREDICTOR from VP]
[V_TRANS -> • attend, origin=2] [PREDICTOR from VP]
[V_TRANS -> • attends, origin=2] [PREDICTOR from VP]
[V_TRANS -> • book, origin=2] [PREDICTOR from VP]
[V_TRANS -> • books, origin=2] [PREDICTOR from VP]
[V_TRANS -> • check, origin=2] [PREDICTOR from VP]
[V_TRANS -> • checks, origin=2] [PREDICTOR from VP]
[V_TRANS -> • need, origin=2] [PREDICTOR from VP]
[V_TRANS -> • needs, origin=2] [PREDICTOR from VP]
[V_TRANS -> • open, origin=2] [PREDICTOR from VP]
[V_TRANS -> • opens, origin=2] [PREDICTOR from VP]
[V_TRANS -> • pay, origin=2] [PREDICTOR from VP]
[V_TRANS -> • pays, origin=2] [PREDICTOR from VP]
[V_TRANS -> • read, origin=2] [PREDICTOR from VP]
[V_TRANS -> • reads, origin=2] [PREDICTOR from VP]
[V_TRANS -> • saw, origin=2] [PREDICTOR from VP]
[V_TRANS -> • show, origin=2] [PREDICTOR from VP]
[V_TRANS -> • shows, origin=2] [PREDICTOR from VP]
[V_TRANS -> • submit, origin=2] [PREDICTOR from VP]
[V_TRANS -> • submits, origin=2] [PREDICTOR from VP]
[V_TRANS -> • submitted, origin=2] [PREDICTOR from VP]
[V_TRANS -> • update, origin=2] [PREDICTOR from VP]
[V_TRANS -> • updates, origin=2] [PREDICTOR from VP]
[V_DITR -> • give, origin=2] [PREDICTOR from VP]
[V_DITR -> • gives, origin=2] [PREDICTOR from VP]
[V_DITR -> • send, origin=2] [PREDICTOR from VP]
[V_DITR -> • sends, origin=2] [PREDICTOR from VP]
[V_DITR -> • sent, origin=2] [PREDICTOR from VP]
[V_DITR -> • show, origin=2] [PREDICTOR from VP]
[V_DITR -> • shows, origin=2] [PREDICTOR from VP]
[V_DITR -> • tell, origin=2] [PREDICTOR from VP]
[V_DITR -> • tells, origin=2] [PREDICTOR from VP]
[V_OBJCOMP -> • ask, origin=2] [PREDICTOR from VP]
[V_OBJCOMP -> • asks, origin=2] [PREDICTOR from VP]
[V_OBJCOMP -> • tell, origin=2] [PREDICTOR from VP]
[V_OBJCOMP -> • tells, origin=2] [PREDICTOR from VP]
[V_SENT -> • know, origin=2] [PREDICTOR from VP]
[V_SENT -> • knows, origin=2] [PREDICTOR from VP]
[V_SENT -> • say, origin=2] [PREDICTOR from VP]
[V_SENT -> • says, origin=2] [PREDICTOR from VP]
[V_WH -> • ask, origin=2] [PREDICTOR from VP]
[V_WH -> • asks, origin=2] [PREDICTOR from VP]
[V_WH -> • tell, origin=2] [PREDICTOR from VP]
[V_WH -> • tells, origin=2] [PREDICTOR from VP]
[V_PP -> • applies, origin=2] [PREDICTOR from VP]
[V_PP -> • apply, origin=2] [PREDICTOR from VP]

[V_PP -> • wait, origin=2] [PREDICTOR from VP]
[V_PP -> • waits, origin=2] [PREDICTOR from VP]
[COP -> • are, origin=2] [PREDICTOR from VP]
[COP -> • be, origin=2] [PREDICTOR from VP]
[COP -> • is, origin=2] [PREDICTOR from VP]
[COP -> • seem, origin=2] [PREDICTOR from VP]
[COP -> • seems, origin=2] [PREDICTOR from VP]
[COP -> • was, origin=2] [PREDICTOR from VP]
[COP -> • were, origin=2] [PREDICTOR from VP]
[AUX -> • are, origin=2] [PREDICTOR from VP]
[AUX -> • can, origin=2] [PREDICTOR from VP]
[AUX -> • could, origin=2] [PREDICTOR from VP]
[AUX -> • did, origin=2] [PREDICTOR from VP]
[AUX -> • do, origin=2] [PREDICTOR from VP]
[AUX -> • does, origin=2] [PREDICTOR from VP]
[AUX -> • has, origin=2] [PREDICTOR from VP]
[AUX -> • have, origin=2] [PREDICTOR from VP]
[AUX -> • is, origin=2] [PREDICTOR from VP]
[AUX -> • may, origin=2] [PREDICTOR from VP]
[AUX -> • might, origin=2] [PREDICTOR from VP]
[AUX -> • should, origin=2] [PREDICTOR from VP]
[AUX -> • was, origin=2] [PREDICTOR from VP]
[AUX -> • were, origin=2] [PREDICTOR from VP]
[AUX -> • will, origin=2] [PREDICTOR from VP]
[AUX -> • would, origin=2] [PREDICTOR from VP]
[PREP -> • about, origin=2] [PREDICTOR from PP]
[PREP -> • after, origin=2] [PREDICTOR from PP]
[PREP -> • at, origin=2] [PREDICTOR from PP]
[PREP -> • before, origin=2] [PREDICTOR from PP]
[PREP -> • by, origin=2] [PREDICTOR from PP]
[PREP -> • during, origin=2] [PREDICTOR from PP]
[PREP -> • for, origin=2] [PREDICTOR from PP]
[PREP -> • from, origin=2] [PREDICTOR from PP]
[PREP -> • in, origin=2] [PREDICTOR from PP]
[PREP -> • near, origin=2] [PREDICTOR from PP]
[PREP -> • of, origin=2] [PREDICTOR from PP]
[PREP -> • on, origin=2] [PREDICTOR from PP]
[PREP -> • to, origin=2] [PREDICTOR from PP]
[PREP -> • under, origin=2] [PREDICTOR from PP]
[PREP -> • with, origin=2] [PREDICTOR from PP]
[RELPRO -> • that, origin=2] [PREDICTOR from REL]
[RELPRO -> • which, origin=2] [PREDICTOR from REL]
[RELPRO -> • who, origin=2] [PREDICTOR from REL]
S[3] (104 states)
[V_TRANS -> submits •, origin=2] [SCANNER submits]
[VP -> V_TRANS • NP, origin=2] [COMPLETER V_TRANS]
[VP -> V_TRANS • NP PP, origin=2] [COMPLETER V_TRANS]
[NP -> • DET NBAR, origin=3] [PREDICTOR from VP]
[NP -> • NBAR, origin=3] [PREDICTOR from VP]
[NP -> • PRON, origin=3] [PREDICTOR from VP]
[NP -> • PN, origin=3] [PREDICTOR from VP]
[NP -> • NP PP, origin=3] [PREDICTOR from VP]
[NP -> • NP REL, origin=3] [PREDICTOR from VP]
[NP -> • NP CONJ NP, origin=3] [PREDICTOR from VP]
[DET -> • a, origin=3] [PREDICTOR from NP]
[DET -> • an, origin=3] [PREDICTOR from NP]
[DET -> • each, origin=3] [PREDICTOR from NP]
[DET -> • my, origin=3] [PREDICTOR from NP]
[DET -> • our, origin=3] [PREDICTOR from NP]
[DET -> • that, origin=3] [PREDICTOR from NP]
[DET -> • the, origin=3] [PREDICTOR from NP]
[DET -> • these, origin=3] [PREDICTOR from NP]
[DET -> • this, origin=3] [PREDICTOR from NP]
[DET -> • those, origin=3] [PREDICTOR from NP]
[DET -> • your, origin=3] [PREDICTOR from NP]
[NBAR -> • N, origin=3] [PREDICTOR from NP]
[NBAR -> • ADJ NBAR, origin=3] [PREDICTOR from NP]
[NBAR -> • N NBAR, origin=3] [PREDICTOR from NP]
[NBAR -> • NUM N, origin=3] [PREDICTOR from NP]
[NBAR -> • NBAR CONJ NBAR, origin=3] [PREDICTOR from NP]
[PRON -> • he, origin=3] [PREDICTOR from NP]
[PRON -> • her, origin=3] [PREDICTOR from NP]
[PRON -> • him, origin=3] [PREDICTOR from NP]
[PRON -> • i, origin=3] [PREDICTOR from NP]
[PRON -> • it, origin=3] [PREDICTOR from NP]
[PRON -> • me, origin=3] [PREDICTOR from NP]
[PRON -> • she, origin=3] [PREDICTOR from NP]
[PRON -> • them, origin=3] [PREDICTOR from NP]
[PRON -> • they, origin=3] [PREDICTOR from NP]

[PRON -> • us, origin=3] [PREDICTOR from NP]
[PRON -> • we, origin=3] [PREDICTOR from NP]
[PRON -> • you, origin=3] [PREDICTOR from NP]
[PN -> • arun, origin=3] [PREDICTOR from NP]
[PN -> • hostel, origin=3] [PREDICTOR from NP]
[PN -> • library, origin=3] [PREDICTOR from NP]
[PN -> • maya, origin=3] [PREDICTOR from NP]
[N -> • account, origin=3] [PREDICTOR from NBAR]
[N -> • application, origin=3] [PREDICTOR from NBAR]
[N -> • book, origin=3] [PREDICTOR from NBAR]
[N -> • campus, origin=3] [PREDICTOR from NBAR]
[N -> • charge, origin=3] [PREDICTOR from NBAR]
[N -> • charges, origin=3] [PREDICTOR from NBAR]
[N -> • class, origin=3] [PREDICTOR from NBAR]
[N -> • course, origin=3] [PREDICTOR from NBAR]
[N -> • deadline, origin=3] [PREDICTOR from NBAR]
[N -> • document, origin=3] [PREDICTOR from NBAR]
[N -> • exam, origin=3] [PREDICTOR from NBAR]
[N -> • exams, origin=3] [PREDICTOR from NBAR]
[N -> • fee, origin=3] [PREDICTOR from NBAR]
[N -> • fees, origin=3] [PREDICTOR from NBAR]
[N -> • form, origin=3] [PREDICTOR from NBAR]
[N -> • forms, origin=3] [PREDICTOR from NBAR]
[N -> • grade, origin=3] [PREDICTOR from NBAR]
[N -> • hostel, origin=3] [PREDICTOR from NBAR]
[N -> • id, origin=3] [PREDICTOR from NBAR]
[N -> • ids, origin=3] [PREDICTOR from NBAR]
[N -> • lab, origin=3] [PREDICTOR from NBAR]
[N -> • late, origin=3] [PREDICTOR from NBAR]
[N -> • library, origin=3] [PREDICTOR from NBAR]
[N -> • map, origin=3] [PREDICTOR from NBAR]
[N -> • noon, origin=3] [PREDICTOR from NBAR]
[N -> • notice, origin=3] [PREDICTOR from NBAR]
[N -> • notices, origin=3] [PREDICTOR from NBAR]
[N -> • office, origin=3] [PREDICTOR from NBAR]
[N -> • password, origin=3] [PREDICTOR from NBAR]
[N -> • payment, origin=3] [PREDICTOR from NBAR]
[N -> • placement, origin=3] [PREDICTOR from NBAR]
[N -> • portal, origin=3] [PREDICTOR from NBAR]
[N -> • receipt, origin=3] [PREDICTOR from NBAR]
[N -> • result, origin=3] [PREDICTOR from NBAR]
[N -> • room, origin=3] [PREDICTOR from NBAR]
[N -> • rooms, origin=3] [PREDICTOR from NBAR]
[N -> • schedule, origin=3] [PREDICTOR from NBAR]
[N -> • scholarship, origin=3] [PREDICTOR from NBAR]
[N -> • seminar, origin=3] [PREDICTOR from NBAR]
[N -> • service, origin=3] [PREDICTOR from NBAR]
[N -> • signal, origin=3] [PREDICTOR from NBAR]
[N -> • staff, origin=3] [PREDICTOR from NBAR]
[N -> • student, origin=3] [PREDICTOR from NBAR]
[N -> • students, origin=3] [PREDICTOR from NBAR]
[N -> • time, origin=3] [PREDICTOR from NBAR]
[ADJ -> • available, origin=3] [PREDICTOR from NBAR]
[ADJ -> • campus, origin=3] [PREDICTOR from NBAR]
[ADJ -> • due, origin=3] [PREDICTOR from NBAR]
[ADJ -> • final, origin=3] [PREDICTOR from NBAR]
[ADJ -> • high, origin=3] [PREDICTOR from NBAR]
[ADJ -> • late, origin=3] [PREDICTOR from NBAR]
[ADJ -> • missing, origin=3] [PREDICTOR from NBAR]
[ADJ -> • new, origin=3] [PREDICTOR from NBAR]
[ADJ -> • online, origin=3] [PREDICTOR from NBAR]
[ADJ -> • open, origin=3] [PREDICTOR from NBAR]
[ADJ -> • postponed, origin=3] [PREDICTOR from NBAR]
[ADJ -> • slow, origin=3] [PREDICTOR from NBAR]
[NUM -> • five, origin=3] [PREDICTOR from NBAR]
[NUM -> • many, origin=3] [PREDICTOR from NBAR]
[NUM -> • one, origin=3] [PREDICTOR from NBAR]
[NUM -> • three, origin=3] [PREDICTOR from NBAR]
[NUM -> • two, origin=3] [PREDICTOR from NBAR]
S[4] (69 states)
[DET -> the •, origin=3] [SCANNER the]
[NP -> DET • NBAR, origin=3] [COMPLETER DET]
[NBAR -> • N, origin=4] [PREDICTOR from NP]
[NBAR -> • ADJ NBAR, origin=4] [PREDICTOR from NP]
[NBAR -> • N NBAR, origin=4] [PREDICTOR from NP]
[NBAR -> • NUM N, origin=4] [PREDICTOR from NP]
[NBAR -> • NBAR CONJ NBAR, origin=4] [PREDICTOR from NP]
[N -> • account, origin=4] [PREDICTOR from NBAR]
[N -> • application, origin=4] [PREDICTOR from NBAR]

[N -> • book, origin=4] [PREDICTOR from NBAR]
[N -> • campus, origin=4] [PREDICTOR from NBAR]
[N -> • charge, origin=4] [PREDICTOR from NBAR]
[N -> • charges, origin=4] [PREDICTOR from NBAR]
[N -> • class, origin=4] [PREDICTOR from NBAR]
[N -> • course, origin=4] [PREDICTOR from NBAR]
[N -> • deadline, origin=4] [PREDICTOR from NBAR]
[N -> • document, origin=4] [PREDICTOR from NBAR]
[N -> • exam, origin=4] [PREDICTOR from NBAR]
[N -> • exams, origin=4] [PREDICTOR from NBAR]
[N -> • fee, origin=4] [PREDICTOR from NBAR]
[N -> • fees, origin=4] [PREDICTOR from NBAR]
[N -> • form, origin=4] [PREDICTOR from NBAR]
[N -> • forms, origin=4] [PREDICTOR from NBAR]
[N -> • grade, origin=4] [PREDICTOR from NBAR]
[N -> • hostel, origin=4] [PREDICTOR from NBAR]
[N -> • id, origin=4] [PREDICTOR from NBAR]
[N -> • ids, origin=4] [PREDICTOR from NBAR]
[N -> • lab, origin=4] [PREDICTOR from NBAR]
[N -> • late, origin=4] [PREDICTOR from NBAR]
[N -> • library, origin=4] [PREDICTOR from NBAR]
[N -> • map, origin=4] [PREDICTOR from NBAR]
[N -> • noon, origin=4] [PREDICTOR from NBAR]
[N -> • notice, origin=4] [PREDICTOR from NBAR]
[N -> • notices, origin=4] [PREDICTOR from NBAR]
[N -> • office, origin=4] [PREDICTOR from NBAR]
[N -> • password, origin=4] [PREDICTOR from NBAR]
[N -> • payment, origin=4] [PREDICTOR from NBAR]
[N -> • placement, origin=4] [PREDICTOR from NBAR]
[N -> • portal, origin=4] [PREDICTOR from NBAR]
[N -> • receipt, origin=4] [PREDICTOR from NBAR]
[N -> • result, origin=4] [PREDICTOR from NBAR]
[N -> • room, origin=4] [PREDICTOR from NBAR]
[N -> • rooms, origin=4] [PREDICTOR from NBAR]
[N -> • schedule, origin=4] [PREDICTOR from NBAR]
[N -> • scholarship, origin=4] [PREDICTOR from NBAR]
[N -> • seminar, origin=4] [PREDICTOR from NBAR]
[N -> • service, origin=4] [PREDICTOR from NBAR]
[N -> • signal, origin=4] [PREDICTOR from NBAR]
[N -> • staff, origin=4] [PREDICTOR from NBAR]
[N -> • student, origin=4] [PREDICTOR from NBAR]
[N -> • students, origin=4] [PREDICTOR from NBAR]
[N -> • time, origin=4] [PREDICTOR from NBAR]
[ADJ -> • available, origin=4] [PREDICTOR from NBAR]
[ADJ -> • campus, origin=4] [PREDICTOR from NBAR]
[ADJ -> • due, origin=4] [PREDICTOR from NBAR]
[ADJ -> • final, origin=4] [PREDICTOR from NBAR]
[ADJ -> • high, origin=4] [PREDICTOR from NBAR]
[ADJ -> • late, origin=4] [PREDICTOR from NBAR]
[ADJ -> • missing, origin=4] [PREDICTOR from NBAR]
[ADJ -> • new, origin=4] [PREDICTOR from NBAR]
[ADJ -> • online, origin=4] [PREDICTOR from NBAR]
[ADJ -> • open, origin=4] [PREDICTOR from NBAR]
[ADJ -> • postponed, origin=4] [PREDICTOR from NBAR]
[ADJ -> • slow, origin=4] [PREDICTOR from NBAR]
[NUM -> • five, origin=4] [PREDICTOR from NBAR]
[NUM -> • many, origin=4] [PREDICTOR from NBAR]
[NUM -> • one, origin=4] [PREDICTOR from NBAR]
[NUM -> • three, origin=4] [PREDICTOR from NBAR]
[NUM -> • two, origin=4] [PREDICTOR from NBAR]
S[5] (109 states)
[N -> late •, origin=4] [SCANNER late]
[ADJ -> late •, origin=4] [SCANNER late]
[NBAR -> N •, origin=4] [COMPLETER N]
[NBAR -> N • NBAR, origin=4] [COMPLETER N]
[NBAR -> ADJ • NBAR, origin=4] [COMPLETER ADJ]
[NP -> DET NBAR •, origin=3] [COMPLETER NBAR]
[NBAR -> NBAR • CONJ NBAR, origin=4] [COMPLETER NBAR]
[NBAR -> • N, origin=5] [PREDICTOR from NBAR]
[NBAR -> • ADJ NBAR, origin=5] [PREDICTOR from NBAR]
[NBAR -> • N NBAR, origin=5] [PREDICTOR from NBAR]
[NBAR -> • NUM N, origin=5] [PREDICTOR from NBAR]
[NBAR -> • NBAR CONJ NBAR, origin=5] [PREDICTOR from NBAR]
[VP -> V_TRANS NP •, origin=2] [COMPLETER NP]
[VP -> V_TRANS NP • PP, origin=2] [COMPLETER NP]
[NP -> NP • PP, origin=3] [COMPLETER NP]
[NP -> NP • REL, origin=3] [COMPLETER NP]
[NP -> NP • CONJ NP, origin=3] [COMPLETER NP]
[CONJ -> • and, origin=5] [PREDICTOR from NBAR]

[CONJ -> • but, origin=5] [PREDICTOR from NBAR]
[CONJ -> • or, origin=5] [PREDICTOR from NBAR]
[N -> • account, origin=5] [PREDICTOR from NBAR]
[N -> • application, origin=5] [PREDICTOR from NBAR]
[N -> • book, origin=5] [PREDICTOR from NBAR]
[N -> • campus, origin=5] [PREDICTOR from NBAR]
[N -> • charge, origin=5] [PREDICTOR from NBAR]
[N -> • charges, origin=5] [PREDICTOR from NBAR]
[N -> • class, origin=5] [PREDICTOR from NBAR]
[N -> • course, origin=5] [PREDICTOR from NBAR]
[N -> • deadline, origin=5] [PREDICTOR from NBAR]
[N -> • document, origin=5] [PREDICTOR from NBAR]
[N -> • exam, origin=5] [PREDICTOR from NBAR]
[N -> • exams, origin=5] [PREDICTOR from NBAR]
[N -> • fee, origin=5] [PREDICTOR from NBAR]
[N -> • fees, origin=5] [PREDICTOR from NBAR]
[N -> • form, origin=5] [PREDICTOR from NBAR]
[N -> • forms, origin=5] [PREDICTOR from NBAR]
[N -> • grade, origin=5] [PREDICTOR from NBAR]
[N -> • hostel, origin=5] [PREDICTOR from NBAR]
[N -> • id, origin=5] [PREDICTOR from NBAR]
[N -> • ids, origin=5] [PREDICTOR from NBAR]
[N -> • lab, origin=5] [PREDICTOR from NBAR]
[N -> • late, origin=5] [PREDICTOR from NBAR]
[N -> • library, origin=5] [PREDICTOR from NBAR]
[N -> • map, origin=5] [PREDICTOR from NBAR]
[N -> • noon, origin=5] [PREDICTOR from NBAR]
[N -> • notice, origin=5] [PREDICTOR from NBAR]
[N -> • notices, origin=5] [PREDICTOR from NBAR]
[N -> • office, origin=5] [PREDICTOR from NBAR]
[N -> • password, origin=5] [PREDICTOR from NBAR]
[N -> • payment, origin=5] [PREDICTOR from NBAR]
[N -> • placement, origin=5] [PREDICTOR from NBAR]
[N -> • portal, origin=5] [PREDICTOR from NBAR]
[N -> • receipt, origin=5] [PREDICTOR from NBAR]
[N -> • result, origin=5] [PREDICTOR from NBAR]
[N -> • room, origin=5] [PREDICTOR from NBAR]
[N -> • rooms, origin=5] [PREDICTOR from NBAR]
[N -> • schedule, origin=5] [PREDICTOR from NBAR]
[N -> • scholarship, origin=5] [PREDICTOR from NBAR]
[N -> • seminar, origin=5] [PREDICTOR from NBAR]
[N -> • service, origin=5] [PREDICTOR from NBAR]
[N -> • signal, origin=5] [PREDICTOR from NBAR]
[N -> • staff, origin=5] [PREDICTOR from NBAR]
[N -> • student, origin=5] [PREDICTOR from NBAR]
[N -> • students, origin=5] [PREDICTOR from NBAR]
[N -> • time, origin=5] [PREDICTOR from NBAR]
[ADJ -> • available, origin=5] [PREDICTOR from NBAR]
[ADJ -> • campus, origin=5] [PREDICTOR from NBAR]
[ADJ -> • due, origin=5] [PREDICTOR from NBAR]
[ADJ -> • final, origin=5] [PREDICTOR from NBAR]
[ADJ -> • high, origin=5] [PREDICTOR from NBAR]
[ADJ -> • late, origin=5] [PREDICTOR from NBAR]
[ADJ -> • missing, origin=5] [PREDICTOR from NBAR]
[ADJ -> • new, origin=5] [PREDICTOR from NBAR]
[ADJ -> • online, origin=5] [PREDICTOR from NBAR]
[ADJ -> • open, origin=5] [PREDICTOR from NBAR]
[ADJ -> • postponed, origin=5] [PREDICTOR from NBAR]
[ADJ -> • slow, origin=5] [PREDICTOR from NBAR]
[NUM -> • five, origin=5] [PREDICTOR from NBAR]
[NUM -> • many, origin=5] [PREDICTOR from NBAR]
[NUM -> • one, origin=5] [PREDICTOR from NBAR]
[NUM -> • three, origin=5] [PREDICTOR from NBAR]
[NUM -> • two, origin=5] [PREDICTOR from NBAR]
[DECL -> NP VP •, origin=0] [COMPLETER VP]
[VP -> VP • PP, origin=2] [COMPLETER VP]
[VP -> VP • CONJ VP, origin=2] [COMPLETER VP]
[PP -> • PREP NP, origin=5] [PREDICTOR from VP]
[REL -> • RELPRO VP, origin=5] [PREDICTOR from NP]
[REL -> • RELPRO AUX V_INTR, origin=5] [PREDICTOR from NP]
[REL -> • RELPRO AUX VP, origin=5] [PREDICTOR from NP]
[S -> DECL •, origin=0] [COMPLETER DECL]
[PREP -> • about, origin=5] [PREDICTOR from PP]
[PREP -> • after, origin=5] [PREDICTOR from PP]
[PREP -> • at, origin=5] [PREDICTOR from PP]
[PREP -> • before, origin=5] [PREDICTOR from PP]
[PREP -> • by, origin=5] [PREDICTOR from PP]
[PREP -> • during, origin=5] [PREDICTOR from PP]
[PREP -> • for, origin=5] [PREDICTOR from PP]

[PREP -> • from, origin=5] [PREDICTOR from PP]
[PREP -> • in, origin=5] [PREDICTOR from PP]
[PREP -> • near, origin=5] [PREDICTOR from PP]
[PREP -> • of, origin=5] [PREDICTOR from PP]
[PREP -> • on, origin=5] [PREDICTOR from PP]
[PREP -> • to, origin=5] [PREDICTOR from PP]
[PREP -> • under, origin=5] [PREDICTOR from PP]
[PREP -> • with, origin=5] [PREDICTOR from PP]
[RELPRO -> • that, origin=5] [PREDICTOR from REL]
[RELPRO -> • which, origin=5] [PREDICTOR from REL]
[RELPRO -> • who, origin=5] [PREDICTOR from REL]
[ROOT -> S •, origin=0] [COMPLETER S]
S[6] (110 states)
[N -> form •, origin=5] [SCANNER form]
[NBAR -> N •, origin=5] [COMPLETER N]
[NBAR -> N • NBAR, origin=5] [COMPLETER N]
[NBAR -> N NBAR •, origin=4] [COMPLETER NBAR]
[NBAR -> ADJ NBAR •, origin=4] [COMPLETER NBAR]
[NBAR -> NBAR • CONJ NBAR, origin=5] [COMPLETER NBAR]
[NBAR -> • N, origin=6] [PREDICTOR from NBAR]
[NBAR -> • ADJ NBAR, origin=6] [PREDICTOR from NBAR]
[NBAR -> • N NBAR, origin=6] [PREDICTOR from NBAR]
[NBAR -> • NUM N, origin=6] [PREDICTOR from NBAR]
[NBAR -> • NBAR CONJ NBAR, origin=6] [PREDICTOR from NBAR]
[NP -> DET NBAR •, origin=3] [COMPLETER NBAR]
[NBAR -> NBAR • CONJ NBAR, origin=4] [COMPLETER NBAR]
[CONJ -> • and, origin=6] [PREDICTOR from NBAR]
[CONJ -> • but, origin=6] [PREDICTOR from NBAR]
[CONJ -> • or, origin=6] [PREDICTOR from NBAR]
[N -> • account, origin=6] [PREDICTOR from NBAR]
[N -> • application, origin=6] [PREDICTOR from NBAR]
[N -> • book, origin=6] [PREDICTOR from NBAR]
[N -> • campus, origin=6] [PREDICTOR from NBAR]
[N -> • charge, origin=6] [PREDICTOR from NBAR]
[N -> • charges, origin=6] [PREDICTOR from NBAR]
[N -> • class, origin=6] [PREDICTOR from NBAR]
[N -> • course, origin=6] [PREDICTOR from NBAR]
[N -> • deadline, origin=6] [PREDICTOR from NBAR]
[N -> • document, origin=6] [PREDICTOR from NBAR]
[N -> • exam, origin=6] [PREDICTOR from NBAR]
[N -> • exams, origin=6] [PREDICTOR from NBAR]
[N -> • fee, origin=6] [PREDICTOR from NBAR]
[N -> • fees, origin=6] [PREDICTOR from NBAR]
[N -> • form, origin=6] [PREDICTOR from NBAR]
[N -> • forms, origin=6] [PREDICTOR from NBAR]
[N -> • grade, origin=6] [PREDICTOR from NBAR]
[N -> • hostel, origin=6] [PREDICTOR from NBAR]
[N -> • id, origin=6] [PREDICTOR from NBAR]
[N -> • ids, origin=6] [PREDICTOR from NBAR]
[N -> • lab, origin=6] [PREDICTOR from NBAR]
[N -> • late, origin=6] [PREDICTOR from NBAR]
[N -> • library, origin=6] [PREDICTOR from NBAR]
[N -> • map, origin=6] [PREDICTOR from NBAR]
[N -> • noon, origin=6] [PREDICTOR from NBAR]
[N -> • notice, origin=6] [PREDICTOR from NBAR]
[N -> • notices, origin=6] [PREDICTOR from NBAR]
[N -> • office, origin=6] [PREDICTOR from NBAR]
[N -> • password, origin=6] [PREDICTOR from NBAR]
[N -> • payment, origin=6] [PREDICTOR from NBAR]
[N -> • placement, origin=6] [PREDICTOR from NBAR]
[N -> • portal, origin=6] [PREDICTOR from NBAR]
[N -> • receipt, origin=6] [PREDICTOR from NBAR]
[N -> • result, origin=6] [PREDICTOR from NBAR]
[N -> • room, origin=6] [PREDICTOR from NBAR]
[N -> • rooms, origin=6] [PREDICTOR from NBAR]
[N -> • schedule, origin=6] [PREDICTOR from NBAR]
[N -> • scholarship, origin=6] [PREDICTOR from NBAR]
[N -> • seminar, origin=6] [PREDICTOR from NBAR]
[N -> • service, origin=6] [PREDICTOR from NBAR]
[N -> • signal, origin=6] [PREDICTOR from NBAR]
[N -> • staff, origin=6] [PREDICTOR from NBAR]
[N -> • student, origin=6] [PREDICTOR from NBAR]
[N -> • students, origin=6] [PREDICTOR from NBAR]
[N -> • time, origin=6] [PREDICTOR from NBAR]
[ADJ -> • available, origin=6] [PREDICTOR from NBAR]
[ADJ -> • campus, origin=6] [PREDICTOR from NBAR]
[ADJ -> • due, origin=6] [PREDICTOR from NBAR]
[ADJ -> • final, origin=6] [PREDICTOR from NBAR]
[ADJ -> • high, origin=6] [PREDICTOR from NBAR]

[ADJ -> • late, origin=6] [PREDICTOR from NBAR]
[ADJ -> • missing, origin=6] [PREDICTOR from NBAR]
[ADJ -> • new, origin=6] [PREDICTOR from NBAR]
[ADJ -> • online, origin=6] [PREDICTOR from NBAR]
[ADJ -> • open, origin=6] [PREDICTOR from NBAR]
[ADJ -> • postponed, origin=6] [PREDICTOR from NBAR]
[ADJ -> • slow, origin=6] [PREDICTOR from NBAR]
[NUM -> • five, origin=6] [PREDICTOR from NBAR]
[NUM -> • many, origin=6] [PREDICTOR from NBAR]
[NUM -> • one, origin=6] [PREDICTOR from NBAR]
[NUM -> • three, origin=6] [PREDICTOR from NBAR]
[NUM -> • two, origin=6] [PREDICTOR from NBAR]
[VP -> V_TRANS NP •, origin=2] [COMPLETER NP]
[VP -> V_TRANS NP • PP, origin=2] [COMPLETER NP]
[NP -> NP • PP, origin=3] [COMPLETER NP]
[NP -> NP • REL, origin=3] [COMPLETER NP]
[NP -> NP • CONJ NP, origin=3] [COMPLETER NP]
[DECL -> NP VP •, origin=0] [COMPLETER VP]
[VP -> VP • PP, origin=2] [COMPLETER VP]
[VP -> VP • CONJ VP, origin=2] [COMPLETER VP]
[PP -> • PREP NP, origin=6] [PREDICTOR from VP]
[REL -> • RELPRO VP, origin=6] [PREDICTOR from NP]
[REL -> • RELPRO AUX V_INTR, origin=6] [PREDICTOR from NP]
[REL -> • RELPRO AUX VP, origin=6] [PREDICTOR from NP]
[S -> DECL •, origin=0] [COMPLETER DECL]
[PREP -> • about, origin=6] [PREDICTOR from PP]
[PREP -> • after, origin=6] [PREDICTOR from PP]
[PREP -> • at, origin=6] [PREDICTOR from PP]
[PREP -> • before, origin=6] [PREDICTOR from PP]
[PREP -> • by, origin=6] [PREDICTOR from PP]
[PREP -> • during, origin=6] [PREDICTOR from PP]
[PREP -> • for, origin=6] [PREDICTOR from PP]
[PREP -> • from, origin=6] [PREDICTOR from PP]
[PREP -> • in, origin=6] [PREDICTOR from PP]
[PREP -> • near, origin=6] [PREDICTOR from PP]
[PREP -> • of, origin=6] [PREDICTOR from PP]
[PREP -> • on, origin=6] [PREDICTOR from PP]
[PREP -> • to, origin=6] [PREDICTOR from PP]
[PREP -> • under, origin=6] [PREDICTOR from PP]
[PREP -> • with, origin=6] [PREDICTOR from PP]
[RELPRO -> • that, origin=6] [PREDICTOR from REL]
[RELPRO -> • which, origin=6] [PREDICTOR from REL]
[RELPRO -> • who, origin=6] [PREDICTOR from REL]
[ROOT -> S •, origin=0] [COMPLETER S]

Appendix D – Full PCFG Production and Probability Table

The next table lists every licensed production, its count in the 20-tree miniature treebank and its add-0.1 smoothed probability. Structural and lexical rules are both included. Zero-count rules remain in support with nonzero probability; probabilities sum to one for each left-hand side.

LHS	RHS	Treebank count	Smoothed probability
S	DECL	20	0.98529412
S	IMP	0	0.00490196
S	YNQ	0	0.00490196
S	WHQ	0	0.00490196
DECL	NP VP	20	1.0
IMP	VP	0	1.0
YNQ	AUX NP VP	0	1.0
WHQ	WH NP VP	0	0.16666667
WHQ	WH AUX NP VP	0	0.16666667
WHQ	WH VP	0	0.16666667
WHQ	WH COP NP	0	0.16666667
WHQ	WH COP ADJP	0	0.16666667
WHQ	WH AUX NP V_TRANS	0	0.16666667
NP	DET NBAR	34	0.92915531
NP	NBAR	1	0.02997275
NP	PRON	0	0.0027248
NP	PN	0	0.0027248
NP	NP PP	1	0.02997275
NP	NP REL	0	0.0027248
NP	NP CONJ NP	0	0.0027248
NBAR	N	35	0.98873239
NBAR	ADJ NBAR	0	0.0028169
NBAR	N NBAR	0	0.0028169
NBAR	NUM N	0	0.0028169
NBAR	NBAR CONJ NBAR	0	0.0028169
VP	V_INTR	6	0.25630252
VP	V_TRANS NP	11	0.46638655
VP	V_DITR NP NP	0	0.00420168
VP	V_OBJCOMP NP S	0	0.00420168
VP	V_OBJCOMP NP THAT S	0	0.00420168
VP	V_SENT THAT S	0	0.00420168
VP	V_WH NP WHCLAUSE	0	0.00420168
VP	V_PP PP	0	0.00420168
VP	COP ADJP	2	0.08823529
VP	COP NP	0	0.00420168
VP	AUX VP	0	0.00420168
VP	AUX V_INTR	0	0.00420168
VP	VP PP	2	0.08823529
VP	VP CONJ VP	0	0.00420168

LHS	RHS	Treebank count	Smoothed probability
VP	V_TRANS NP PP	0	0.00420168
VP	V_DITR NP PP	0	0.00420168
VP	V_INTR PP	1	0.04621849
VP	V_INTR CONJ VP	0	0.00420168
PP	PREP NP	4	1.0
REL	RELPRO VP	0	0.33333333
REL	RELPRO AUX V_INTR	0	0.33333333
REL	RELPRO AUX VP	0	0.33333333
WHCLAUSE	WH NP VP	0	0.33333333
WHCLAUSE	WH VP	0	0.33333333
WHCLAUSE	WHETHER S	0	0.33333333
ADJP	ADJ	2	0.91304348
ADJP	ADJ PP	0	0.04347826
ADJP	ADJP CONJ ADJP	0	0.04347826
DET	a	1	0.03133903
DET	an	0	0.002849
DET	each	0	0.002849
DET	my	0	0.002849
DET	our	0	0.002849
DET	that	0	0.002849
DET	the	33	0.94301994
DET	these	0	0.002849
DET	this	0	0.002849
DET	those	0	0.002849
DET	your	0	0.002849
PRON	he	0	0.08333333
PRON	her	0	0.08333333
PRON	him	0	0.08333333
PRON	i	0	0.08333333
PRON	it	0	0.08333333
PRON	me	0	0.08333333
PRON	she	0	0.08333333
PRON	them	0	0.08333333
PRON	they	0	0.08333333
PRON	us	0	0.08333333
PRON	we	0	0.08333333
PRON	you	0	0.08333333
PN	arun	0	0.25
PN	hostel	0	0.25
PN	library	0	0.25
PN	maya	0	0.25
N	account	0	0.00253165
N	application	0	0.00253165

LHS	RHS	Treebank count	Smoothed probability
N	book	1	0.0278481
N	campus	0	0.00253165
N	charge	0	0.00253165
N	charges	0	0.00253165
N	class	0	0.00253165
N	course	1	0.0278481
N	deadline	0	0.00253165
N	document	1	0.0278481
N	exam	0	0.00253165
N	exams	0	0.00253165
N	fee	2	0.05316456
N	fees	0	0.00253165
N	form	3	0.07848101
N	forms	0	0.00253165
N	grade	0	0.00253165
N	hostel	0	0.00253165
N	id	0	0.00253165
N	ids	0	0.00253165
N	lab	0	0.00253165
N	late	0	0.00253165
N	library	4	0.10379747
N	map	1	0.0278481
N	noon	0	0.00253165
N	notice	3	0.07848101
N	notices	0	0.00253165
N	office	0	0.00253165
N	password	0	0.00253165
N	payment	0	0.00253165
N	placement	0	0.00253165
N	portal	3	0.07848101
N	receipt	0	0.00253165
N	result	0	0.00253165
N	room	3	0.07848101
N	rooms	0	0.00253165
N	schedule	0	0.00253165
N	scholarship	0	0.00253165
N	seminar	3	0.07848101
N	service	0	0.00253165
N	signal	0	0.00253165
N	staff	1	0.0278481
N	student	7	0.17974684
N	students	2	0.05316456
N	time	0	0.00253165

LHS	RHS	Treebank count	Smoothed probability
ADJ	available	0	0.03125
ADJ	campus	0	0.03125
ADJ	due	2	0.65625
ADJ	final	0	0.03125
ADJ	high	0	0.03125
ADJ	late	0	0.03125
ADJ	missing	0	0.03125
ADJ	new	0	0.03125
ADJ	online	0	0.03125
ADJ	open	0	0.03125
ADJ	postponed	0	0.03125
ADJ	slow	0	0.03125
NUM	five	0	0.2
NUM	many	0	0.2
NUM	one	0	0.2
NUM	three	0	0.2
NUM	two	0	0.2
V_INTR	arrived	0	0.01176471
V_INTR	crashed	1	0.12941176
V_INTR	crashes	1	0.12941176
V_INTR	expire	0	0.01176471
V_INTR	expired	0	0.01176471
V_INTR	open	0	0.01176471
V_INTR	opened	0	0.01176471
V_INTR	opens	3	0.36470588
V_INTR	postponed	0	0.01176471
V_INTR	sleep	0	0.01176471
V_INTR	starts	1	0.12941176
V_INTR	wait	0	0.01176471
V_INTR	waits	0	0.01176471
V_INTR	work	0	0.01176471
V_INTR	works	1	0.12941176
V_TRANS	accept	0	0.00746269
V_TRANS	accepts	0	0.00746269
V_TRANS	attend	0	0.00746269
V_TRANS	attends	1	0.08208955
V_TRANS	book	0	0.00746269
V_TRANS	books	2	0.15671642
V_TRANS	check	0	0.00746269
V_TRANS	checks	0	0.00746269
V_TRANS	need	0	0.00746269
V_TRANS	needs	0	0.00746269
V_TRANS	open	0	0.00746269

LHS	RHS	Treebank count	Smoothed probability
V_TRANS	opens	0	0.00746269
V_TRANS	pay	0	0.00746269
V_TRANS	pays	0	0.00746269
V_TRANS	read	1	0.08208955
V_TRANS	reads	3	0.23134328
V_TRANS	saw	0	0.00746269
V_TRANS	show	0	0.00746269
V_TRANS	shows	0	0.00746269
V_TRANS	submit	1	0.08208955
V_TRANS	submits	3	0.23134328
V_TRANS	submitted	0	0.00746269
V_TRANS	update	0	0.00746269
V_TRANS	updates	0	0.00746269
V_DITR	give	0	0.11111111
V_DITR	gives	0	0.11111111
V_DITR	send	0	0.11111111
V_DITR	sends	0	0.11111111
V_DITR	sent	0	0.11111111
V_DITR	show	0	0.11111111
V_DITR	shows	0	0.11111111
V_DITR	tell	0	0.11111111
V_DITR	tells	0	0.11111111
V_OBJCOMP	ask	0	0.25
V_OBJCOMP	asks	0	0.25
V_OBJCOMP	tell	0	0.25
V_OBJCOMP	tells	0	0.25
V_SENT	know	0	0.25
V_SENT	knows	0	0.25
V_SENT	say	0	0.25
V_SENT	says	0	0.25
V_WH	ask	0	0.25
V_WH	asks	0	0.25
V_WH	tell	0	0.25
V_WH	tells	0	0.25
V_PP	applies	0	0.25
V_PP	apply	0	0.25
V_PP	wait	0	0.25
V_PP	waits	0	0.25
AUX	are	0	0.0625
AUX	can	0	0.0625
AUX	could	0	0.0625
AUX	did	0	0.0625
AUX	do	0	0.0625

LHS	RHS	Treebank count	Smoothed probability
AUX	does	0	0.0625
AUX	has	0	0.0625
AUX	have	0	0.0625
AUX	is	0	0.0625
AUX	may	0	0.0625
AUX	might	0	0.0625
AUX	should	0	0.0625
AUX	was	0	0.0625
AUX	were	0	0.0625
AUX	will	0	0.0625
AUX	would	0	0.0625
COP	are	0	0.03703704
COP	be	0	0.03703704
COP	is	2	0.77777778
COP	seem	0	0.03703704
COP	seems	0	0.03703704
COP	was	0	0.03703704
COP	were	0	0.03703704
PREP	about	0	0.01818182
PREP	after	0	0.01818182
PREP	at	0	0.01818182
PREP	before	0	0.01818182
PREP	by	0	0.01818182
PREP	during	0	0.01818182
PREP	for	1	0.2
PREP	from	0	0.01818182
PREP	in	0	0.01818182
PREP	near	2	0.38181818
PREP	of	0	0.01818182
PREP	on	0	0.01818182
PREP	to	0	0.01818182
PREP	under	0	0.01818182
PREP	with	1	0.2
CONJ	and	0	0.33333333
CONJ	but	0	0.33333333
CONJ	or	0	0.33333333
WH	how	0	0.16666667
WH	what	0	0.16666667
WH	when	0	0.16666667
WH	where	0	0.16666667
WH	who	0	0.16666667
WH	why	0	0.16666667
WHETHER	whether	0	1.0

LHS	RHS	Treebank count	Smoothed probability
RELPRO	that	0	0.33333333
RELPRO	which	0	0.33333333
RELPRO	who	0	0.33333333
THAT	that	0	1.0

Appendix E – PCFG Ambiguity, CKY and Feature-Test Outputs

The following raw generated outputs expose the exact competing tree scores and the CKY span calculations used in Task 9.

NP-attachment: (S (IMP (VP (V_TRANS book) (NP (NP (DET the) (NBAR (N room))) (PP (PREP near) (NP (DET the) (NBAR (N library))))))))

Probability = 1.1937724509e-09; selected tree = VP-attachment

VP-attachment: (S (IMP (VP (VP (V_TRANS book) (NP (DET the) (NBAR (N room)))) (PP (PREP near) (NP (DET the) (NBAR (N library)))))))

Probability = 3.51428734878e-09; selected tree = VP-attachment

CNF-equivalent subgrammar (composed unit-rule probabilities):
S -> NP VP: 0.9852941176
NP -> DET N: 0.9186859577
VP -> crashed: 0.03316856154
CKY Viterbi chart for “the portal crashed” (span: category = best probability):
[0:1] the: DET=0.943019943
[1:2] portal: N=0.07848101266
[2:3] crashed: VP=0.03316856154
[0:2] the portal: NP=0.06799117611
[0:3] the portal crashed: S=0.002222005252

Unification test output

plural NP with plural VP: {"AGR": {"NUM": "pl", "PER": 3}} ⊔ {"AGR": {"NUM": "pl", "PER": 3}} = {"AGR": {"NUM": "pl", "PER": 3}} [PASS]

plural NP with singular VP: {"AGR": {"NUM": "pl", "PER": 3}} ⊔ {"AGR": {"NUM": "sg", "PER": 3}} = FAIL [PASS]

nominative subject with nominative slot: {"CASE": "nom", "NUM": "sg"} ⊔ {"CASE": "nom"} = {"CASE": "nom", "NUM": "sg"} [PASS]

accusative object with nominative subject slot: {"CASE": "acc", "NUM": "sg"} ⊔ {"CASE": "nom"} = FAIL [PASS]

finite present subject agrees in number/person: {"AGR": {"NUM": "sg", "PER": 3}} ⊔ {"AGR": {"NUM": "sg", "PER": 3}, "TENSE": "pres"} = {"AGR": {"NUM": "sg", "PER": 3}, "TENSE": "pres"} [PASS]

Appendix F – All 30 Benchmark Records

The next table is the machine-readable result in a compact one-row-per-input form. “Best outputs” is 1 when a Viterbi parse exists, not the number of all PCFG parses. Parse counts are exact up to the enumeration cap of 100.

ID	Tok.	Type	TD parses	TD edges	Earley parses	Earley edges	Viterbi states	Viterbi candidates
T01	3	declarative	1	101	1	101	16	37
T02	5	declarative	1	141	1	141	23	47
T03	4	declarative	1	132	1	132	18	38
T04	3	imperative	1	94	1	94	12	24
T05	6	yes_no	1	177	1	177	27	54
T06	5	wh_object	1	186	1	186	15	24
T07	4	wh_subject	1	133	1	133	19	39
T08	4	wh_locative	2	182	2	182	18	39
T09	5	wh_temporal	2	198	2	198	28	75
T10	4	imperative_ditransitive	1	118	1	118	15	29
T11	7	sentential_complement	1	296	1	296	34	67
T12	5	pp_complement	1	125	1	125	27	57
T13	5	ditransitive	1	140	1	140	23	48
T14	5	declarative_pp	2	159	2	159	32	85
T15	8	ambiguous_np_vp_pp	11	283	11	283	58	184
T16	9	ambiguous_multiple_pp	8	245	8	245	49	136
T17	7	ambiguous_pp	3	181	3	181	34	77
T18	7	declarative_np_pp	1	200	1	200	34	69
T19	7	coordination	2	203	2	203	34	75
T20	6	yes_no	1	177	1	177	28	56
T21	7	relative_clause	1	261	1	261	36	71
T22	4	passive_like	3	159	3	159	24	67
T23	7	embedded_wh	4	279	4	279	39	100
T24	6	adjective_phrase	2	172	2	172	39	92
T25	20	longer_pp_20_tokens	92	480	92	480	178	497
T26	5	ungrammatical_agreement	1	141	1	141	23	47
T27	5	ungrammatical_det_number	1	141	1	141	23	47
T28	4	ungrammatical_pronoun_case	1	124	1	124	17	34
T29	1	ungrammatical_subcat	0	60	0	60	2	2
T30	5	ungrammatical_subcat	0	103	0	103	23	50

Full sentences and recorded times

ID	Input sentence	Top-down ms	Earley ms	Viterbi ms
T01	the portal crashed	1.8041	1.8529	3.4578
T02	the student submits the form	2.226	2.6226	6.6681
T03	students submit the form	2.1494	2.4168	4.596
T04	submit the form	1.5441	1.6463	2.7334
T05	does the student submit the form	2.8678	3.9355	8.9935
T06	what does the student submit	2.9425	3.2401	4.976
T07	who submits the form	2.1043	2.1833	4.3582
T08	where is the exam	2.9931	3.1138	4.4006
T09	when will the library open	3.2626	4.0976	6.9235
T10	send me the receipt	57.607	2.1989	4.1368
T11	tell me that the fee is due	5.1424	6.282	11.7793
T12	apply for a hostel room	2.2655	2.5235	7.6545
T13	show the student the notice	2.3473	3.6402	6.7581
T14	the library opens at noon	2.6902	2.7569	8.0769
T15	show the students with the late fee notice	5.3629	5.7201	20.6676
T16	book the room near the library for the seminar	4.649	4.4837	22.887
T17	i saw the notice with a map	3.4531	3.4121	12.9301
T18	the students with ids submit the form	3.6334	3.4271	12.2851
T19	the staff and students attend the seminar	3.8974	3.5659	12.4677
T20	does the hostel accept the application	3.1875	4.8062	8.7252
T21	the fee that is due is high	3.9824	4.5742	11.6912
T22	the exams were postponed	2.9024	3.0609	5.4237
T23	tell me when the exams will open	4.4091	6.3905	12.3388
T24	the student submits the late form	3.001	3.017	12.254
T25	the student in my hostel submitted the late form after the final exam at the campus library with the staff	9.521	9.1572	136.8089
T26	the students submits the form	2.3008	2.5535	6.8229
T27	this students submit the form	2.346	2.4126	6.8931
T28	me submits the form	2.2806	3.4805	4.367
T29	submit	1.0777	0.9732	0.3275
T30	the portal crashed the form	1.7005	1.9278	6.4749

Appendix G – Day-4 Coverage Test Results

The CSV-generated evidence for the original-grammar checks is repeated here for review. The first uncovered token is the first lexical barrier encountered by the grammar's coverage check; all OOV items are listed.

ID	Message	All OOV words	Result
D1	Hi, can you tell me when the exams that were postponed will be held, and also whether the hostel fee is due?	hi; held; also	coverage error: first OOV=hi
D2	Hey, I missed the exam because my bus was delayed. Can you tell me the new timetable?	hey; missed; because; bus; delayed; timetable	coverage error: first OOV=hey
D3	fees? hostel too, please check my payment.	too; please	coverage error: first OOV=too

Projected modified-system handling is summarized as partial analyses rather than claimed full parses: D1 yields an initial YNQ and an embedded when-clause candidate, followed by a coordinated whether-clause; D2 yields a causal-clause boundary and a second request; D3 yields two fragments and an imperative request. These are target structures for a future regression suite, not outputs of the current baseline parser.

Appendix H – Twenty Hand-Annotated Miniature Training Trees

These are the 20 trees used to estimate the PCFG. They were manually constructed for the assignment, normalized to the declared CFG, and verified to contain only licensed productions. They are not sampled from a corpus and are not a gold-standard treebank.

01. (S (NP (DET the) (N portal)) (VP (V crashed)))
02. (S (NP (DET the) (N student)) (VP (V submits) (NP (DET the) (N form))))
03. (S (NP (N students)) (VP (V submit) (NP (DET the) (N form))))
04. (S (NP (DET the) (N library)) (VP (V opens)))
05. (S (NP (DET the) (N fee)) (VP (V is) (NP (ADJ due))))
06. (S (NP (DET the) (N room)) (VP (V books) (NP (DET the) (N seminar))))
07. (S (NP (DET the) (N student)) (VP (V reads) (NP (DET the) (N notice))))
08. (S (NP (DET the) (N staff)) (VP (V attends) (NP (DET the) (N seminar))))
09. (S (NP (DET the) (N portal)) (VP (V works)))
10. (S (NP (DET the) (N student)) (VP (VP (V books) (NP (DET the) (N room))) (PP (P near) (NP (DET the) (N library)))))
11. (S (NP (DET the) (N student)) (VP (V submits) (NP (NP (DET the) (N form)) (PP (P for) (NP (DET the) (N course))))))
12. (S (NP (DET the) (N room)) (VP (V opens) (PP (P near) (NP (DET the) (N library)))))
13. (S (NP (DET the) (N student)) (VP (V reads) (NP (DET the) (N book))))
14. (S (NP (DET the) (N fee)) (VP (V is) (NP (ADJ due))))
15. (S (NP (DET the) (N portal)) (VP (V crashes)))
16. (S (NP (DET the) (N student)) (VP (V submits) (NP (DET the) (N document))))
17. (S (NP (DET the) (N library)) (VP (V opens)))
18. (S (NP (DET the) (N students)) (VP (V read) (NP (DET the) (N notice))))
19. (S (NP (DET the) (N student)) (VP (VP (V reads) (NP (DET the) (N notice))) (PP (P with) (NP (DET a) (N map)))))
20. (S (NP (DET the) (N seminar)) (VP (V starts)))

Appendix I – Reproducible Experiment Source Code

The complete source follows. It creates all CSVs, traces and figures used in this report. The script is also provided separately in the supplementary package. To rerun it, install NLTK and Matplotlib, then execute the script from the package directory.

```
#!/usr/bin/env python3
"""Reproducible CFG/Earley/PCFG exercises for DSA03 NLP Assignment 3.


The experiments use a deliberately controlled English helpdesk fragment. It is
not intended to be a broad-coverage English parser. Fixed seed: 42.
"""
from __future__ import annotations
import csv, json, math, os, random, re, time
from collections import Counter, defaultdict, deque
from pathlib import Path
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import nltk
from nltk import CFG, PCFG, Nonterminal, Tree
from nltk.parse import EarleyChartParser, TopDownChartParser, ViterbiParser
from nltk.grammar import Production, ProbabilisticProduction


OUT=Path(os.environ.get('ASSIGNMENT3_OUT',Path(__file__).resolve().parent/'assignment3_artifacts'))
OUT.mkdir(parents=True,exist_ok=True)
SEED=42
random.seed(SEED)


# More than 40 phrase-structure productions; lexical productions are stored separately.
STRUCT=[
('S',('DECL',)),('S',('IMP',)),('S',('YNQ',)),('S',('WHQ',)),
('DECL',('NP','VP')),('IMP',('VP',)),('YNQ',('AUX','NP','VP')),
('WHQ',('WH','NP','VP')),('WHQ',('WH','AUX','NP','VP')),
('WHQ',('WH','VP')),('WHQ',('WH','COP','NP')),
('WHQ',('WH','COP','ADJP')),('WHQ',('WH','AUX','NP','V_TRANS')),
('NP',('DET','NBAR')),('NP',('NBAR',)),('NP',('PRON',)),('NP',('PN',)),
('NP',('NP','PP')),('NP',('NP','REL')),('NP',('NP','CONJ','NP')),
('NBAR',('N',)),('NBAR',('ADJ','NBAR')),('NBAR',('N','NBAR')),
('NBAR',('NUM','N')),('NBAR',('NBAR','CONJ','NBAR')),
('VP',('V_INTR',)),('VP',('V_TRANS','NP')),('VP',('V_DITR','NP','NP')),
('VP',('V_OBJCOMP','NP','S')),('VP',('V_OBJCOMP','NP','THAT','S')),
('VP',('V_SENT','THAT','S')),
('VP',('V_WH','NP','WHCLAUSE')),('VP',('V_PP','PP')),
('VP',('COP','ADJP')),('VP',('COP','NP')),('VP',('AUX','VP')),
('VP',('AUX','V_INTR')),('VP',('VP','PP')),('VP',('VP','CONJ','VP')),
('VP',('V_TRANS','NP','PP')),('VP',('V_DITR','NP','PP')),
('VP',('V_INTR','PP')),('VP',('V_INTR','CONJ','VP')),
('PP',('PREP','NP')),('REL',('RELPRO','VP')),
('REL',('RELPRO','AUX','V_INTR')),('REL',('RELPRO','AUX','VP')),
('WHCLAUSE',('WH','NP','VP')),('WHCLAUSE',('WH','VP')),
('WHCLAUSE',('WHETHER','S')),('ADJP',('ADJ',)),
('ADJP',('ADJ','PP')),('ADJP',('ADJP','CONJ','ADJP')),
]
LEX={
'DET':['the','a','an','this','that','these','those','my','your','our','each'],
'PRON':['i','you','he','she','it','we','they','me','him','her','us','them'],
'PN':['maya','arun','library','hostel'],
'N':
['portal','student','students','form','forms','room','rooms','library','seminar','fee','fees','notice','notices','exam','exams','hostel','application','receipt','signal','staff','map','campus','
id','ids','charge','charges','time','office','book','class','course','result','payment','account','schedule','grade','lab','deadline','document','password','scholarship','placement','servic
e','late','noon'],
'ADJ':['late','due','slow','open','available','postponed','high','new','missing','final','online','campus'],
'NUM':['one','two','three','five','many'],
'V_INTR':['crashed','crashes','arrived','opened','open','opens','sleep','wait','waits','postponed','expire','expired','work','works','starts'],
'V_TRANS':
['submit','submits','submitted','book','books','read','reads','saw','attend','attends','accept','accepts','pay','pays','check','checks','update','updates','need','needs','open','opens','sho
w','shows'],
'V_DITR':['send','sends','sent','give','gives','show','shows','tell','tells'],
'V_OBJCOMP':['tell','tells','ask','asks'],
'V_SENT':['know','knows','say','says'],
'V_WH':['tell','tells','ask','asks'],
'V_PP':['apply','applies','wait','waits'],
'AUX':['do','does','did','will','can','could','would','should','may','might','is','are','was','were','has','have'],
'COP':['is','are','was','were','be','seems','seem'],
'PREP':['with','near','for','in','at','after','before','by','to','from','on','under','during','about','of'],
'CONJ':['and','or','but'],
'WH':['what','who','where','when','why','how'],
'WHETHER':['whether'],
'RELPRO':['that','who','which'],
```

```
'THAT':['that'],
}
# Keep lexical productions independent of phrase rules.
LEX_PRODS=[]
for lhs,words in LEX.items():
    for word in sorted(set(words)):
        LEX_PRODS.append((lhs,(word,)))
PRODS=[Production(Nonterminal(a),[Nonterminal(x) if x in LEX or x in {r[0] for r in STRUCT} else x for x in b]) for a,b in STRUCT]
for a,b in LEX_PRODS:
    PRODS.append(Production(Nonterminal(a),list(b)))
CFG_GRAMMAR=CFG(Nonterminal('S'),PRODS)


# 30-sentence controlled evaluation set. The five starred strings are deliberately ill-formed.
TESTS=[
('T01','the portal crashed','declarative'),
('T02','the student submits the form','declarative'),
('T03','students submit the form','declarative'),
('T04','submit the form','imperative'),
('T05','does the student submit the form','yes_no'),
('T06','what does the student submit','wh_object'),
('T07','who submits the form','wh_subject'),
('T08','where is the exam','wh_locative'),
('T09','when will the library open','wh_temporal'),
('T10','send me the receipt','imperative_ditransitive'),
('T11','tell me that the fee is due','sentential_complement'),
('T12','apply for a hostel room','pp_complement'),
('T13','show the student the notice','ditransitive'),
('T14','the library opens at noon','declarative_pp'),
('T15','show the students with the late fee notice','ambiguous_np_vp_pp'),
('T16','book the room near the library for the seminar','ambiguous_multiple_pp'),
('T17','i saw the notice with a map','ambiguous_pp'),
('T18','the students with ids submit the form','declarative_np_pp'),
('T19','the staff and students attend the seminar','coordination'),
('T20','does the hostel accept the application','yes_no'),
('T21','the fee that is due is high','relative_clause'),
('T22','the exams were postponed','passive_like'),
('T23','tell me when the exams will open','embedded_wh'),
('T24','the student submits the late form','adjective_phrase'),
('T25','the student in my hostel submitted the late form after the final exam at the campus library with the staff','longer_pp_20_tokens'),
('T26','the students submits the form','ungrammatical_agreement'),
('T27','this students submit the form','ungrammatical_det_number'),
('T28','me submits the form','ungrammatical_pronoun_case'),
('T29','submit','ungrammatical_subcat'),
('T30','the portal crashed the form','ungrammatical_subcat'),
]


def toks(s): return re.findall(r"[a-z]+(?:'[a-z]+)?|[0-9]+",s.lower())


def writecsv(name,headers,rows):
    with (OUT/name).open('w',newline='',encoding='utf-8') as f:
        w=csv.writer(f);w.writerow(headers);w.writerows(rows)


# Simple explicit feature filter for the finite agreement test set.
GOOD_AGR=[
('the student submits the form',True),('students submit the form',True),
('i submit the form',True),('they submit the form',True),
('the portal crashes',True),('the forms are due',True),
('the students submits the form',False),('the student submit the form',False),
('this students submit the form',False),('these student submits the form',False),
('me submits the form',False),('they is due',False),
]
SG_N={'student','portal','form','room','library','seminar','fee','notice','exam','hostel','application','receipt','signal','map','campus','office','course','result','payment','account','sc
hedule','grade','lab','deadline','document','password','scholarship','placement','service'}
PL_N={'students','forms','rooms','fees','notices','exams','ids','charges','courses','results'}
SG_V={'submits','crashes','opens','arrives','attends','accepts','pays','checks','updates','needs','is','was','has','seems'}
PL_V={'submit','crash','open','arrive','attend','accept','pay','check','update','need','are','were','have','seem'}
SUBJ={'i','you','he','she','it','we','they','me','him','her','us','them'}
OBJ={'me','him','her','us','them','you'}


def agreement_ok(s):
    w=toks(s)
    # Controlled mini-checks for the explicit test suite; not a general English analyzer.
    if not w:return False
    for i,x in enumerate(w):
        if x in {'this','that','each'} and i+1<len(w) and w[i+1] in PL_N:return False
        if x in {'these','those'} and i+1<len(w) and w[i+1] in SG_N:return False
    subj=None;verb=None
    # Find the first nominal subject and first finite verb in the clause.
    for i,x in enumerate(w):
        if x in SG_N|PL_N|SUBJ and subj is None:subj=x
        if x in SG_V|PL_V and verb is None:verb=x
```

```
    if subj is not None and verb is not None:
        plural=(subj in PL_N or subj in {'we','they'})
        if subj in {'i','you'}: plural=None
        if plural is True and verb in SG_V:return False
        if plural is False and verb in PL_V:return False
        if subj in {'me','him','her','us','them'} and w[0]==subj:return False
    return True


agr_rows=[]
for s,expected in GOOD_AGR:
    baseline=bool(list(EarleyChartParser(CFG_GRAMMAR).parse(toks(s))))
    aware=baseline and agreement_ok(s)
    agr_rows.append((s,expected,int(baseline),int(aware),'PASS' if aware==expected else 'CHECK'))
writecsv('agreement_tests.csv',['sentence','gold_grammatical','baseline_cfg_accepts','feature_filter_accepts','check'],agr_rows)


# Subcategorization diagnostics: the grammar contains category-specific frames.
SUBCAT_EXPECT=[
('the portal crashed',True,'intransitive'),('submit the form',True,'transitive imperative'),
('send me the receipt',True,'ditransitive'),('tell me that the fee is due',True,'object + clause'),
('apply for a hostel room',True,'PP complement'),('submit',False,'missing required NP'),
('the portal crashed the form',False,'intransitive verb over-complemented'),
('the student sent the receipt',False,'ditransitive verb under-complemented'),
]
subcat_rows=[]
for s,exp,frame in SUBCAT_EXPECT:
    parses=list(EarleyChartParser(CFG_GRAMMAR).parse(toks(s)))
    # apply for ... is analyzed by V_PP + PP; other examples use frame-specific VP rules.
    ok=bool(parses)
    subcat_rows.append((s,frame,int(ok),int(exp),'PASS' if ok==exp else 'CHECK'))
writecsv('subcategorization_tests.csv',['sentence','expected_frame','baseline_cfg_accepts','expected_accept','check'],subcat_rows)


# Earley chart recognizer with complete item provenance for a hand simulation.
def earley_chart(sentence):
    words=toks(sentence); n=len(words)
    rules=defaultdict(list)
    for p in CFG_GRAMMAR.productions():
        rhs=tuple(str(x) if isinstance(x,Nonterminal) else x for x in p.rhs())
        rules[str(p.lhs())].append(rhs)
    nt=set(rules)
    charts=[[] for _ in range(n+1)]; seen=[set() for _ in range(n+1)]; source={}
    start=('ROOT',('S',),0,0)
    def add(i,state,op):
        if state not in seen[i]:
            seen[i].add(state);charts[i].append(state);source[(i,state)]=op;return True
        return False
    add(0,start,'initialize')
    # ROOT -> S is a synthetic start production.
    rules['ROOT']=[('S',)]
    for i in range(n+1):
        k=0
        while k<len(charts[i]):
            st=charts[i][k];lhs,rhs,dot,origin=st;k+=1
            if dot<len(rhs):
                sym=rhs[dot]
                if sym in nt:
                    for rr in rules[sym]:add(i,(sym,rr,0,i),f'PREDICTOR from {lhs}')
                    # Nullable grammar is absent; this guard only handles already-completed zero-width states.
                    for done in list(charts[i]):
                        if done[0]==sym and done[2]==len(done[1]) and done[3]==i:
                            add(i,(lhs,rhs,dot+1,origin),f'COMPLETER from {sym}')
                elif i<n and words[i]==sym:
                    add(i+1,(lhs,rhs,dot+1,origin),f'SCANNER {sym}')
            else:
                for parent in list(charts[origin]):
                    pl,pr,pd,po=parent
                    if pd<len(pr) and pr[pd]==lhs:
                        add(i,(pl,pr,pd+1,po),f'COMPLETER {lhs}')
    accepted=('ROOT',('S',),1,0) in seen[n]
    return words,charts,source,accepted


CHART_SENT='the student submits the late form'
words,charts,source,accepted=earley_chart(CHART_SENT)
with (OUT/'earley_hand_chart.txt').open('w',encoding='utf-8') as f:
    f.write(f"Sentence: {CHART_SENT}\nTokens: {words}\nAccepted: {accepted}\n\n")
    for i,chart in enumerate(charts):
        f.write(f'S[{i}] ({len(chart)} states)\n')
        for st in chart:
            lhs,rhs,dot,origin=st;marked=list(rhs);marked.insert(dot,'•')
            f.write(f"  [{lhs} -> {' '.join(marked)}, origin={origin}]  [{source[(i,st)]}]\n")


# 20 hand-annotated miniature trees for MLE PCFG estimation.
```

```
TREEBANK=[
'(S (NP (DET the) (N portal)) (VP (V crashed)))',
'(S (NP (DET the) (N student)) (VP (V submits) (NP (DET the) (N form))))',
'(S (NP (N students)) (VP (V submit) (NP (DET the) (N form))))',
'(S (NP (DET the) (N library)) (VP (V opens)))',
'(S (NP (DET the) (N fee)) (VP (V is) (NP (ADJ due))))',
'(S (NP (DET the) (N room)) (VP (V books) (NP (DET the) (N seminar))))',
'(S (NP (DET the) (N student)) (VP (V reads) (NP (DET the) (N notice))))',
'(S (NP (DET the) (N staff)) (VP (V attends) (NP (DET the) (N seminar))))',
'(S (NP (DET the) (N portal)) (VP (V works)))',
'(S (NP (DET the) (N student)) (VP (VP (V books) (NP (DET the) (N room))) (PP (P near) (NP (DET the) (N library)))))',
'(S (NP (DET the) (N student)) (VP (V submits) (NP (NP (DET the) (N form)) (PP (P for) (NP (DET the) (N course))))))',
'(S (NP (DET the) (N room)) (VP (V opens) (PP (P near) (NP (DET the) (N library)))))',
'(S (NP (DET the) (N student)) (VP (V reads) (NP (DET the) (N book))))',
'(S (NP (DET the) (N fee)) (VP (V is) (NP (ADJ due))))',
'(S (NP (DET the) (N portal)) (VP (V crashes)))',
'(S (NP (DET the) (N student)) (VP (V submits) (NP (DET the) (N document))))',
'(S (NP (DET the) (N library)) (VP (V opens)))',
'(S (NP (DET the) (N students)) (VP (V read) (NP (DET the) (N notice))))',
'(S (NP (DET the) (N student)) (VP (VP (V reads) (NP (DET the) (N notice))) (PP (P with) (NP (DET a) (N map)))))',
'(S (NP (DET the) (N seminar)) (VP (V starts)))',
]
def normalize_demo_tree(t):
    """Map compact hand-written trees into productions of the declared CFG."""
    if not isinstance(t,Tree): return t
    children=[normalize_demo_tree(c) if isinstance(c,Tree) else c for c in t]
    label=t.label()
    if label=='P': label='PREP'
    if label=='NP':
        if len(children)==2 and isinstance(children[0],Tree) and children[0].label()=='DET' and isinstance(children[1],Tree) and children[1].label()=='N':
            children=[children[0],Tree('NBAR',[children[1]])]
        elif len(children)==1 and isinstance(children[0],Tree) and children[0].label()=='N':
            children=[Tree('NBAR',[children[0]])]
        elif len(children)==1 and isinstance(children[0],Tree) and children[0].label()=='ADJ':
            return Tree('ADJP',children)
    if label=='VP' and children and isinstance(children[0],Tree) and children[0].label()=='V':
        word=children[0][0]
        if word in {'is','are','was','were','be'}:
            children[0]=Tree('COP',[word])
        elif word in {'submits','submit','books','reads','read','attends'}:
            children[0]=Tree('V_TRANS',[word])
        else:
            children[0]=Tree('V_INTR',[word])
    if label=='S' and len(children)==2 and isinstance(children[0],Tree) and children[0].label()=='NP' and isinstance(children[1],Tree) and children[1].label()=='VP':
        return Tree('S',[Tree('DECL',children)])
    return Tree(label,children)


TREES=[normalize_demo_tree(Tree.fromstring(x)) for x in TREEBANK]
counts=Counter(p for t in TREES for p in t.productions())
grammar_prod_set=set(CFG_GRAMMAR.productions())
off_grammar=[str(p) for p in counts if p not in grammar_prod_set]
if off_grammar: raise ValueError('Treebank productions not licensed by CFG: '+repr(off_grammar[:10]))
lhs_counts=Counter()
for p,c in counts.items():lhs_counts[p.lhs()]+=c
alpha=0.1
pcfg_rows=[];probs={}
prod_by_lhs=defaultdict(list)
for p in CFG_GRAMMAR.productions():prod_by_lhs[p.lhs()].append(p)
for lhs,alts in prod_by_lhs.items():
    denom=lhs_counts[lhs]+alpha*len(alts)
    for p in alts:
        rhs=tuple(str(x) if isinstance(x,Nonterminal) else x for x in p.rhs())
        count=counts.get(p,0);prob=(count+alpha)/denom
        probs[(str(lhs),rhs)]=prob
        pcfg_rows.append((str(lhs),' '.join(rhs),count,round(prob,8)))
writecsv('pcfg_mle_rules.csv',['lhs','rhs','treebank_count','smoothed_probability'],pcfg_rows)
# Build full PCFG over the CFG structure with treebank MLE counts and alpha smoothing.
# Rule support includes all structural and lexical productions, so each LHS sums to one.
full_counts=counts
pcfg_prods=[]
for lhs,alts in prod_by_lhs.items():
    total=sum(full_counts[p] for p in alts);den=total+alpha*len(alts)
    for p in alts:
        pcfg_prods.append(ProbabilisticProduction(p.lhs(),p.rhs(),prob=(full_counts[p]+alpha)/den))
PCFG_GRAMMAR=PCFG(Nonterminal('S'),pcfg_prods)
# Check normalization for every left-hand side.
prob_sums=defaultdict(float)
for p in PCFG_GRAMMAR.productions():prob_sums[p.lhs()]+=p.prob()


def ruleprob(lhs,rhs): return probs[(lhs,tuple(rhs))]
def treeprob(tree):
    val=1.0
    for p in tree.productions():
```

```
        rhs=tuple(str(x) if isinstance(x,Nonterminal) else x for x in p.rhs())
        val*=ruleprob(str(p.lhs()),rhs)
    return val

# Unit-rule closure and lexical substitution produce this equivalent CNF subgrammar.
# S=>DECL and DECL=>NP VP; NP=>DET NBAR and NBAR=>N; VP=>V_INTR and V_INTR=>'crashed'.
cnf_s=ruleprob('S',('DECL',))*ruleprob('DECL',('NP','VP'))
cnf_np=ruleprob('NP',('DET','NBAR'))*ruleprob('NBAR',('N',))
cnf_vp=ruleprob('VP',('V_INTR',))*ruleprob('V_INTR',('crashed',))
cky_cells=[
('0:1','the',{'DET':ruleprob('DET',('the',))}),
('1:2','portal',{'N':ruleprob('N',('portal',))}),
('2:3','crashed',{'VP':cnf_vp}),
('0:2','the portal',{'NP':cnf_np*ruleprob('DET',('the',))*ruleprob('N',('portal',))}),
('0:3','the portal crashed',{'S':cnf_s*cnf_np*ruleprob('DET',('the',))*ruleprob('N',('portal',))*cnf_vp}),
]
with (OUT/'cky_trace.txt').open('w',encoding='utf-8') as f:
    f.write('CNF-equivalent subgrammar (composed unit-rule probabilities):\n')
    f.write(f'S -> NP VP: {cnf_s:.10g}\nNP -> DET N: {cnf_np:.10g}\nVP -> crashed: {cnf_vp:.10g}\n')
    f.write('CKY Viterbi chart for “the portal crashed” (span: category = best probability):\n')
    for span,phrase,entries in cky_cells:
        f.write(f'[{span}] {phrase}: '+', '.join(f'{k}={v:.10g}' for k,v in entries.items())+'\n')

def np_the_n(word): return Tree('NP',[Tree('DET',['the']),Tree('NBAR',[Tree('N',[word])])])
base_room=np_the_n('room');np_library=np_the_n('library')
pp_near=Tree('PP',[Tree('PREP',['near']),np_library])
np_attach=Tree('NP',[base_room,pp_near])
tree_np_attach=Tree('S',[Tree('IMP',[Tree('VP',[Tree('V_TRANS',['book']),np_attach])])])
vp_base=Tree('VP',[Tree('V_TRANS',['book']),base_room])
tree_vp_attach=Tree('S',[Tree('IMP',[Tree('VP',[vp_base,pp_near])])])
ambiguity_probs=[treeprob(tree_np_attach),treeprob(tree_vp_attach)]
writecsv('pcfg_ambiguity.csv',['analysis','tree','tree_probability','sum_of_two_trees','viterbi_choice'],[
('NP-attachment',tree_np_attach.pformat(margin=120),f'{ambiguity_probs[0]:.12g}',f'{sum(ambiguity_probs):.12g}','NP-attachment' if
ambiguity_probs[0]>=ambiguity_probs[1] else 'VP-attachment'),
('VP-attachment',tree_vp_attach.pformat(margin=120),f'{ambiguity_probs[1]:.12g}',f'{sum(ambiguity_probs):.12g}','NP-attachment' if
ambiguity_probs[0]>=ambiguity_probs[1] else 'VP-attachment')])

def unify(a,b):
    if isinstance(a,dict) and isinstance(b,dict):
        out={}
        for k in set(a)|set(b):
            if k in a and k in b:
                v=unify(a[k],b[k])
                if v is None:return None
                out[k]=v
            else:out[k]=a[k] if k in a else b[k]
        return out
    return a if a==b else None
UNIFY_CASES=[
('plural NP with plural VP',{'AGR':{'NUM':'pl','PER':3}},{'AGR':{'NUM':'pl','PER':3}},True),
('plural NP with singular VP',{'AGR':{'NUM':'pl','PER':3}},{'AGR':{'NUM':'sg','PER':3}},False),
('nominative subject with nominative slot',{'CASE':'nom','NUM':'sg'},{'CASE':'nom'},True),
('accusative object with nominative subject slot',{'CASE':'acc','NUM':'sg'},{'CASE':'nom'},False),
('finite present subject agrees in number/person',{'AGR':{'NUM':'sg','PER':3}},{'TENSE':'pres','AGR':{'NUM':'sg','PER':3}},True),
]
unify_rows=[]
for name,a,b,expected in UNIFY_CASES:
    result=unify(a,b);ok=(result is not None)
    unify_rows.append((name,json.dumps(a,sort_keys=True),json.dumps(b,sort_keys=True),json.dumps(result,sort_keys=True) if result is not None else
'FAIL',int(ok),int(expected),'PASS' if ok==expected else 'CHECK'))
writecsv('feature_unification_tests.csv',['case','left_avm','right_avm','unified_avm','unifies','expected','check'],unify_rows)

# Parse metrics on 30 items. Tree enumeration is capped at 100 to avoid combinatorial output.
def parse_count(chart, start, cap=100):
    c=0
    for _ in chart.parses(start):
        c+=1
        if c>=cap:return f'>={cap}'
    return c

def run_chart(parser_cls, grammar, sent):
    p=parser_cls(grammar)
    t0=time.perf_counter()
    chart=p.chart_parse(sent)
    elapsed=(time.perf_counter()-t0)*1000
    edges=chart.num_edges() if hasattr(chart,'num_edges') else len(list(chart.edges()))
    pc=parse_count(chart,grammar.start())
    return pc,edges,elapsed,chart

class InstrumentedViterbi(ViterbiParser):
```

```
    def __init__(self,grammar):
        super().__init__(grammar);self.instantiations=0
    def _find_instantiations(self,span,constituents):
        found=super()._find_instantiations(span,constituents)
        self.instantiations+=len(found)
        return found
    def parse_with_counts(self,tokens):
        tokens=list(tokens);self._grammar.check_coverage(tokens);self._parse_deadline=None
        constituents={}
        for i,token in enumerate(tokens):constituents[i,i+1,token]=token
        for length in range(1,len(tokens)+1):
            for start in range(len(tokens)-length+1):
                self._add_constituents_spanning((start,start+length),constituents,tokens)
        tree=constituents.get((0,len(tokens),self._grammar.start()))
        return tree,len(constituents),self.instantiations


metrics=[]
for ident,text,kind in TESTS:
    sent=toks(text)
    topc,tope,topt,_=run_chart(TopDownChartParser,CFG_GRAMMAR,sent)
    eac,eae,eat,_=run_chart(EarleyChartParser,CFG_GRAMMAR,sent)
    vp=InstrumentedViterbi(PCFG_GRAMMAR);t0=time.perf_counter();vbest,vstates,vsteps=vp.parse_with_counts(sent);vt=(time.perf_counter()-t0)*1000
    metrics.append((ident,text,len(sent),kind,topc,tope,round(topt,4),eac,eae,round(eat,4),int(vbest is not None),int(vbest is not None),vstates,vsteps,round(vt,4)))
writecsv('parser_metrics.csv',
['id','sentence','tokens','type','topdown_parse_count_cap100','topdown_chart_edges','topdown_ms','earley_parse_count_cap100','earley_chart_edges','earley_ms','pcfg_pars
e_outputs_best_only','pcfg_best_found','pcfg_chart_constituents','pcfg_rule_instantiations','pcfg_ms'],metrics)
# Parser-state plot by sentence length, averaging over test set.
lengths=sorted(set(r[2] for r in metrics))
plt.figure(figsize=(6.5,4.2))
for label,idx in [('Top-down chart',5),('Earley chart',8)]:
    ys=[]
    for n in lengths:
        vals=[r[idx] for r in metrics if r[2]==n]
        ys.append(sum(vals)/len(vals))
    plt.plot(lengths,ys,marker='o',label=label)
plt.xlabel('Sentence length (tokens)');plt.ylabel('Mean chart edges');plt.title('Parser work on 30 controlled
queries');plt.legend();plt.tight_layout();plt.savefig(OUT/'parser_states.png',dpi=180);plt.close()


# Detailed top-down trace on an unambiguous and an ambiguous item, via chart states.
def chart_trace(parser_cls, sent, name):
    p=parser_cls(CFG_GRAMMAR); chart=p.chart_parse(toks(sent));
    rows=[]
    for edge in chart.edges():
        rows.append(str(edge))
    return f'{name}: {sent}\nEdges={len(rows)}\n'+'\n'.join(rows[:120])+'\n'
with (OUT/'topdown_traces.txt').open('w',encoding='utf-8') as f:
    f.write(chart_trace(TopDownChartParser,'the portal crashed','Unambiguous trace'))
    f.write('\n'+chart_trace(TopDownChartParser,'show the students with the late fee notice','Ambiguous PP attachment trace'))


def show_tree(node,prefix='',last=True):
    label=node.label() if isinstance(node,Tree) else str(node)
    line=prefix+('└── ' if last else '├── ')+label
    if not isinstance(node,Tree):return [line]
    children=list(node);out=[line]
    for i,c in enumerate(children):out.extend(show_tree(c,prefix+('    ' if last else '│   '),i==len(children)-1))
    return out
def derive(tree,right=False):
    items=[tree];forms=[' '.join(x.label() if isinstance(x,Tree) else str(x) for x in items)]
    while any(isinstance(x,Tree) for x in items):
        i=max(j for j,x in enumerate(items) if isinstance(x,Tree)) if right else min(j for j,x in enumerate(items) if isinstance(x,Tree))
        items=items[:i]+list(items[i])+items[i+1:]
        forms.append(' '.join(x.label() if isinstance(x,Tree) else str(x) for x in items))
    return forms
SAMPLE_SENTENCES=['the portal crashed','the student submits the form','submit the form','does the student submit the form','tell me that the fee is due']
with (OUT/'derivations_and_trees.txt').open('w',encoding='utf-8') as f:
    for sent in SAMPLE_SENTENCES:
        tr=next(EarleyChartParser(CFG_GRAMMAR).parse(toks(sent)))
        f.write(f'Sentence: {sent}\nBracketed tree: {tr.pformat(margin=140)}\n')
        category_nodes=len(list(tr.subtrees()));terminal_nodes=len(tr.leaves())
        f.write(f'Height (levels including root): {tr.height()}\nCategory nodes: {category_nodes}; terminal leaves: {terminal_nodes}; total nodes:
{category_nodes+terminal_nodes}; rule applications: {category_nodes}\n')
        f.write('Tree diagram:\n'+'\n'.join(show_tree(tr))+'\n')
        f.write('Leftmost derivation:\n'+'\n'.join(f'  {i+1}. {x}' for i,x in enumerate(derive(tr)))+'\n')
        f.write('Rightmost derivation:\n'+'\n'.join(f'  {i+1}. {x}' for i,x in enumerate(derive(tr,True)))+'\n\n')
with (OUT/'ambiguous_parse_trees.txt').open('w',encoding='utf-8') as f:
    for sent in ['show the students with the late fee notice','book the room near the library for the seminar','i saw the notice with a map']:
        parses=[]
        for i,tr in enumerate(EarleyChartParser(CFG_GRAMMAR).parse(toks(sent))):
            parses.append(tr)
            if i==2:break
        f.write(f'{sent}\nShowing {len(parses)} of the available parses.\n')
        for i,tr in enumerate(parses,1):f.write(f'Parse {i}: {tr.pformat(margin=140)}\n')
```

```
        f.write('\n')


DAY4_MESSAGES=[
('D1','Hi, can you tell me when the exams that were postponed will be held, and also whether the hostel fee is due?'),
('D2','Hey, I missed the exam because my bus was delayed. Can you tell me the new timetable?'),
('D3','fees? hostel too, please check my payment.'),
]
vocab={w for words in LEX.values() for w in words}
day4_rows=[]
for ident,text in DAY4_MESSAGES:
    words=toks(text);oov=[w for w in words if w not in vocab]
    if oov:
        status=f'coverage error: first OOV={oov[0]}'
        parse_count_value='not attempted (coverage error)'
    else:
        chart=EarleyChartParser(CFG_GRAMMAR).chart_parse(words)
        count=parse_count(chart,CFG_GRAMMAR.start())
        status='accepted' if count else 'no complete S parse'
        parse_count_value=count
    day4_rows.append((ident,text,' '.join(words),'; '.join(oov) if oov else 'none',status,parse_count_value))
writecsv('day4_original_tests.csv',['id','message','tokens','all_oov_tokens','original_cfg_result','parse_count_or_note'],day4_rows)


# Full structural grammar and lexicon inventories for the report.
with (OUT/'grammar_inventory.txt').open('w',encoding='utf-8') as f:
    f.write(f'Phrase-structure rules: {len(STRUCT)}\nUnique lexicon words: {len({w for _,ws in LEX.items() for w in ws})}\n\n')
    for a,b in STRUCT:f.write(f'{a} -> {" ".join(b)}\n')
    f.write('\nLEXICON BY CATEGORY\n')
    for a,ws in LEX.items():f.write(f'{a}: {", ".join(sorted(set(ws)))}\n')


# Reproducibility summary and hand-reference checks.
summary={
 'seed':SEED,'structural_rule_count':len(STRUCT),'unique_lexicon_words':len({w for _,ws in LEX.items() for w in ws}),
 'mini_treebank_sentences':len(TREES),'pcfg_lhs_max_sum_error':max(abs(v-1) for v in prob_sums.values()),
 'parser_items':len(TESTS),'agreement_tests':len(agr_rows),'agreement_all_pass':all(x[-1]=='PASS' for x in agr_rows),
 'subcat_tests':len(subcat_rows),'subcat_all_pass':all(x[-1]=='PASS' for x in subcat_rows),
 'day4_messages_tested':len(day4_rows),'day4_original_coverage_failures':sum('OOV' in x[4] for x in day4_rows),
 'earley_hand_sentence':CHART_SENT,'earley_hand_accepted':accepted,'earley_chart_state_counts':[len(c) for c in charts],
 'parser_metrics':metrics,
}
(OUT/'summary.json').write_text(json.dumps(summary,indent=2),encoding='utf-8')
with (OUT/'run_summary.txt').open('w',encoding='utf-8') as f:json.dump(summary,f,indent=2)
print(json.dumps({k:v for k,v in summary.items() if k!='parser_metrics'},indent=2))
print('Wrote results to',OUT)
```

Appendix J – Focused Worked Examples

Worked item	Input	Rule / diagnostic
Intransitive declarative	The portal crashed.	S → DECL → NP VP; VP → V_INTR. The intransitive frame is satisfied without an object.
Transitive imperative	Submit the form.	S → IMP → VP → V_TRANS NP. The understood subject is implicit; the selected object NP is still required.
Yes/no inversion	Does the student submit the form?	S → YNQ → AUX NP VP. Auxiliary precedes subject; the main verb is in base form.
Wh-object extraction	What does the student submit?	WHQ → WH AUX NP V_TRANS. The object is a gap in the compact CFG template; a GAP feature is needed across embedding.
Subject wh-question	Who submits the form?	WHQ → WH VP. The wh phrase occupies subject position and does not need auxiliary inversion here.
Ditransitive frame	Send me the receipt.	VP → V_DITR NP NP. Removing one object violates this implemented frame; other English alternations need separate lexical rules.
PP attachment ambiguity	I saw the notice with a map.	PP may attach under NP or VP; the grammar generates three trees due to interacting category and attachment alternatives.
Left recursion	The student in the hostel submits.	NP → NP PP is left-recursive. Chart-state deduplication terminates; naive recursive descent can loop before consuming input.
Agreement unification	The students submit the form.	Shared AGR features unify for NUM=pl and PER=3; replacing submit with submits creates a conflict.
Case filter	Me submits the form.	The baseline CFG covers the words; CASE=nom on a finite subject rejects the objective pronoun in this context.
Fragment fallback	Fees? Hostel too.	Baseline S rejects bare NP fragments. MSG → FRAG retains partial structure and should attach a low-confidence flag.
Coordinated message	Can you tell me when exams open, and whether the fee is due?	Add coordinated message clauses and embedded wh/whether clauses; rank the forest and use partial parsing when punctuation or agreement is missing.

Across these examples, the parser should retain local structure it can justify, expose ambiguity and uncertainty, and avoid converting a missing rule into a confident wrong intent. The table is a compact regression checklist: every new grammar rule should be accompanied by at least one positive case and one nearby negative or ambiguity case.

References

[1] Earley, J. (1970). “An Efficient Context-Free Parsing Algorithm.” Communications of the ACM, 13(2), 94–102. https://doi.org/10.1145/362007.362035

[2] NLTK Project. “Building Feature Based Grammars,” Natural Language Processing with Python, Chapter 9. https://www.nltk.org/book/ch09.html

[3] Jurafsky, D. and Martin, J. Speech and Language Processing, online draft, Chapter C: Statistical Constituency Parsing. https://web.stanford.edu/~jurafsky/slp3/old_jan23/C.pdf

[4] NLTK Project. “Sample usage for parse.” https://www.nltk.org/howto/parse.html

[5] Shieber, S. M. (1985). “Evidence against the Context-Freeness of Natural Language.” Linguistics and Philosophy, 8, 333–343. https://eecs.harvard.edu/~shieber/Biblio/Papers/shieber85.pdf

[6] NLTK Project. “Sample usage for feature structures.” https://www.nltk.org/howto/featstruct.html

[7] Assignment prompt: DSA03_NLP_Assignment-3.pdf, uploaded by the user; answer format: DSA03-CO2AT3.pdf, uploaded by the user.

Document typeface: Times New Roman throughout. Page count is validated from the rendered PDF preview; the requested editable Word file remains the primary deliverable.